

1.0 Requirements Analysis

1.1 Scope of Work

The system aims to facilitate staffs in the restaurant's point of sales system; the system can help in menu displaying, order taking, payment processing, discount management, and transaction management at the counter, kiosks, and even online order. This system can also be integrated with potential future modules such as inventory management or delivery service from another third party.

The system shall allow customers to place orders via counter, kiosks, or the dedicated application for pickup and delivery. In every order, the customer shall be able to modify their desired orders. The system will track the members' orders and provide them with membership points according to their purchases.

Staffs should be able to update the restaurant's menus by adding, modifying, or removing items on the menu. Staffs can also add promotional discounts and create vouchers. Each item would be categorised into respective categories and have details such as name, description, price, nutritional information. Manager at each branch can dictate what items go on their menu.

The POS system shall let customers pay in various payment methods and provide receipts to the customer once the payment is verified.

Order tracking shall be implemented to let staffs and customers know the progress of the orders that have been confirmed. This allows everyone to be on track on the flow of orders. A simple inventory management system shall update the item's availability in each branch automatically after every order's confirmation. In the occasions that orders are cancelled, staffs are given the ability to edit the availability of the item manually.

Customers shall be able to view their account information, previous purchases, owned vouchers, and membership details such as membership tiers or loyalty points. For the company's end, key details of customers such as orders history are stored to allow the company to tailor services and strengthen customer engagement.

Business Rules

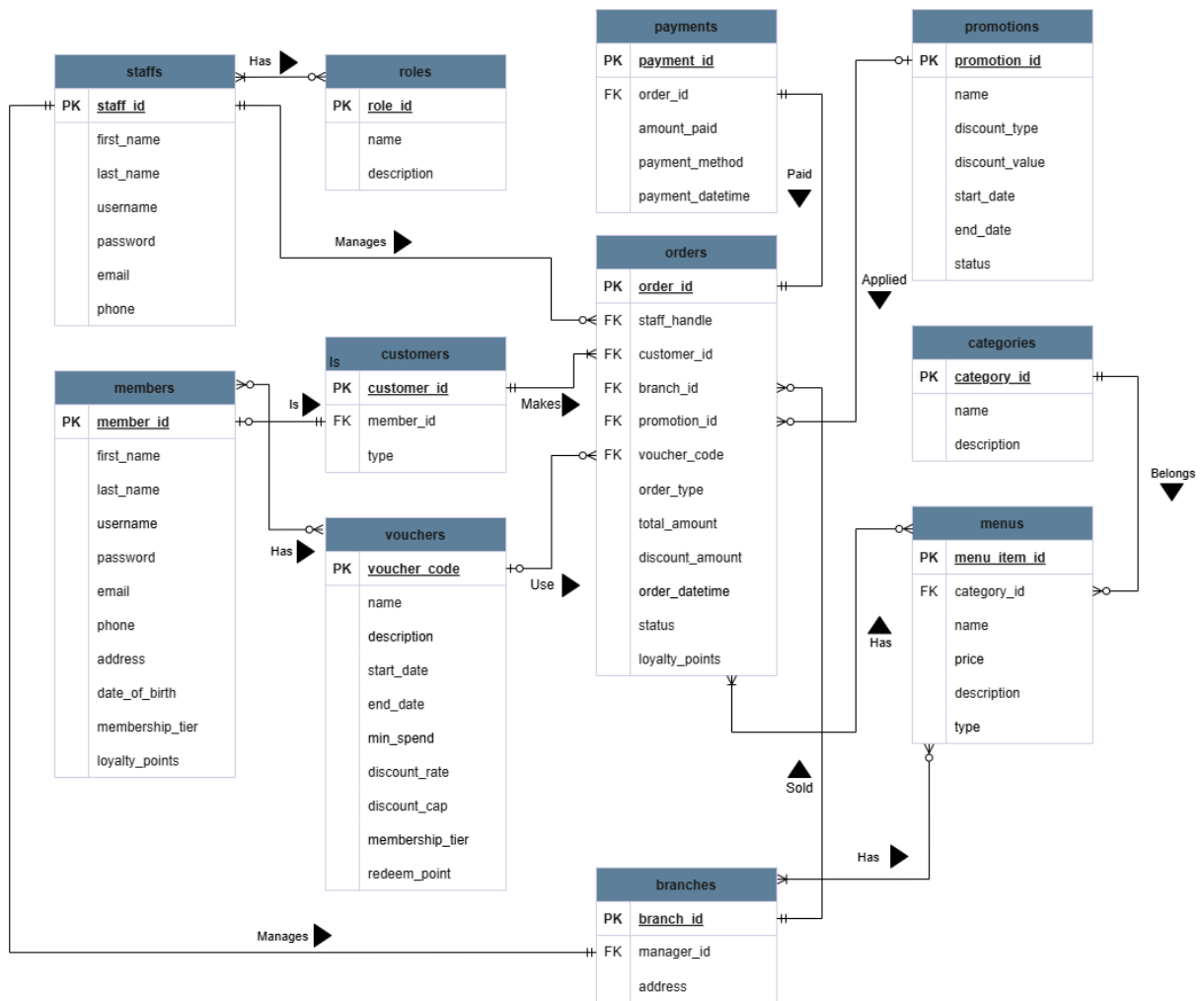
1. One staff can have multiple roles, and one role can be played by multiple staffs.
2. Each staff can manage zero or many orders.
3. Each customer makes one or many orders.
4. Each order originates from exactly one branch.
5. Each customer can only have one and only one membership account.
6. Some customers may not be members.
7. A customer must be 12 or above to sign up for a membership.
8. The membership of each customer is based on the amount spent in the fast-food restaurant.
9. The lowest membership tier is 1 and the highest membership tier is 6.
10. Only members can make orders online.
11. Each member can have none or a lot of vouchers.
12. Each voucher can be owned by none or a lot of members.
13. Each voucher can be used in a lot of orders, but each order can only have one voucher redemption.
14. Each voucher has a minimum membership tier to be utilised.

15. Each voucher only lasts for a specific period.
16. Each order can have none or one promotional discount.
17. Each promotion only lasts for a specific period.
18. Each promotional discount can be applied in multiple orders.
19. Each branch is managed by one staff.
20. Each branch has one or many items on the menu.
21. Each item can belong to one or more branches.
22. The quantity of any item within an order must be a positive integer.
23. All monetary values must be a non-negative.
24. An order's status must follow a sequence – Pending → Confirmed → Preparing → Completed/Cancelled – but not all transitions are possible for every status.
25. An order is considered fully paid only when the sum of associated payments equals or exceeds the total amount of the order
26. The system must only accept payments using a predefined list of acceptable payment methods – Cash, Credit Card, Debit Card, Mobile Payment, and Gift Card

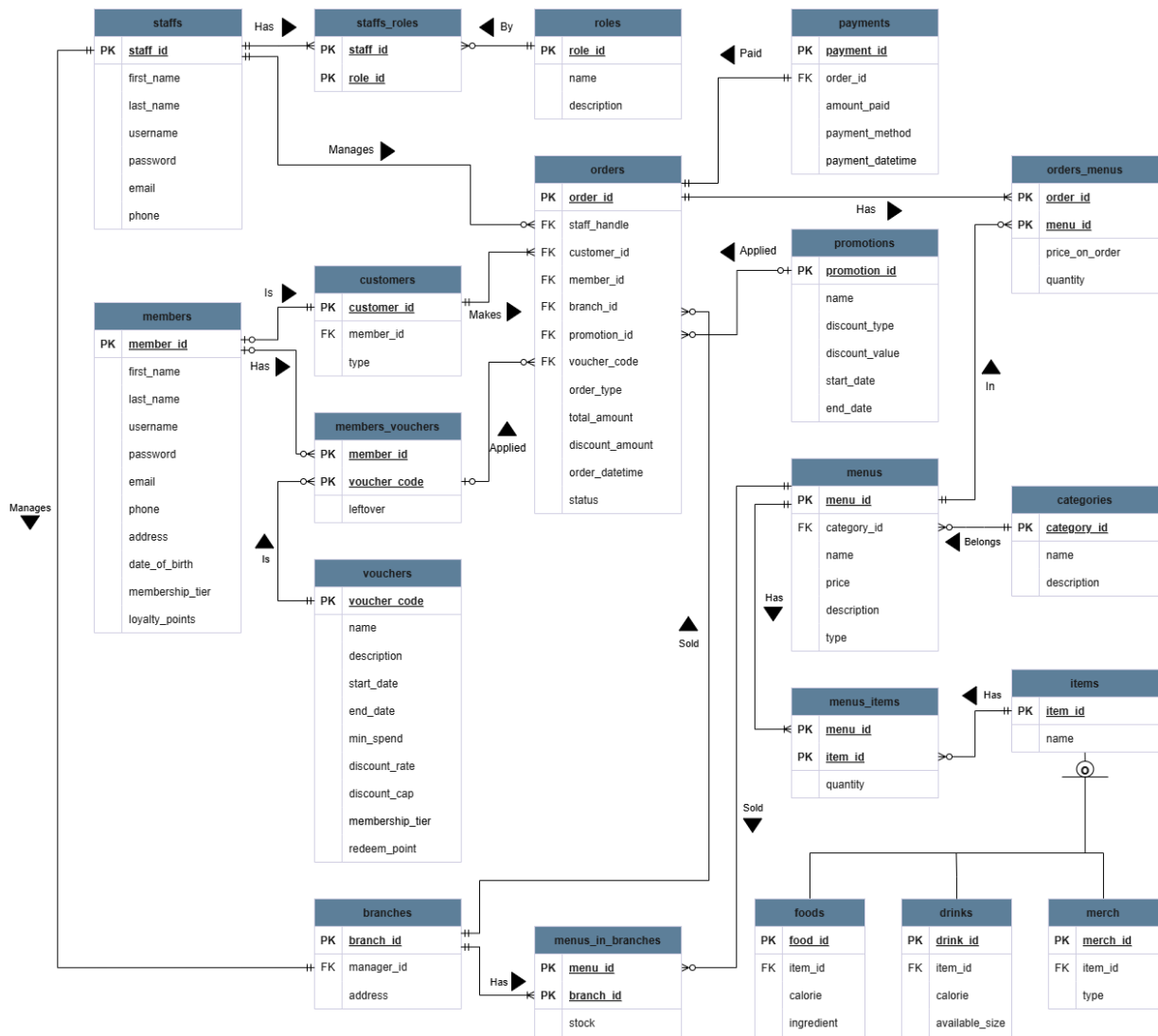
Assumptions

1. All the branches are situated in Malaysia and transactions are done in Malaysian Ringgit.
2. The language used are the languages spoken in Malaysia.
3. The restaurant has a defined menu with fixed items and prices, albeit customisations are possible.
4. There is a system in place that checks stock in real time.
5. Delivery is done from a third-party company.
6. The necessary point of sales hardware peripherals is available and properly configured at each sales counter and kiosk.
7. A straightforward tax model is sufficient, and complex tax scenarios are out of scope.
8. The system's inventory management is limited to tracking current stock levels and reducing them upon sale and does not extend to managing stock levels.
9. Assume a system has been put in place that allows only the managers are allowed to modify the availability of the items in the branch.
10. Assume that an application logic ensures that each order must be confirmed by a staff before preparing the order.
11. Assume that the application provides a security mechanism allowing only authorised personnel to manage and update the prices for menu items.
12. Assume that the usability and intuitiveness of the user interface are handled by the frontend design and does not cause performance constraints on the database schema.
13. Assume the application logic handles the process of converting customer loyalty points to voucher records in the database according to defined business rules for redemption.

1.2 ER Model



1.3 EER Model



1.4 Data Dictionary

ENTITY NAME: STAFFS

Attribute	Data Type	Field Size	Null / Not Null	Constraints	Default Value	References	
staff_id (PK)	VARCHAR2	6	Not Null	Primary Key	-	-	The each
first_name	VARCHAR2	20	Not Null	-	-	-	The staff
last_name	VARCHAR2	20	Not Null	-	-	-	The staff
username	VARCHAR2	20	Not Null	-	-	-	The for th
password	VARCHER2	20	Not Null	At least 8 characters long with one uppercase letter, one symbol, and one digit. Historical password cannot be reused.	Foodie@1233456	-	The used autho
email	VARCHAR2	50	Null	Unique Must follow the format: xxx@xx.xxx	-	-	The email
phone	VARCHAR2	10	Not Null	Only 9 to 10 digits. Malaysian phone number. Start with 01	-	-	The the s

ENTITY NAME: ROLES

ATTRIBUTE	Data Type	Field Size	Null / Not Null	Constraints	Default Value	References
role_id (PK)	VARCHAR2	6	Not Null	Primary key	-	-
name	VARCHAR2	20	Not Null	-	-	-
description	VARCHAR2	1000	Null	-	-	-

ENTITY NAME: MEMBERS

Attribute	Data Type	Field Size	Null / Not Null	Constraints	Default Value	References
member_id (PK)	NUMBER	-	Not Null	Primary key	-	-
first_name	VARCHAR2	20	Not Null	-	-	-
last_name	VARCHAR2	20	Not Null	-	-	-
username	VARCHAR2	20	Not Null	-	-	-
password	VARCHAR2	15	Not Null	At least 8 characters long	-	-

				with one uppercase letter, one symbol, and one digit.			m a
				Historical password cannot be reused.			
email	VARCHAR2	50	Null	Unique Must follow the format: xxx@xx.xxx	-	-	T th
phone	VARCHAR2	10	Not Null	Only 9 to 10 digits. Malaysian phone number. Start with 01	-	-	T o
address	VARCHAR2	300	Null	-	-	-	T a m
date_of_birth	DATE	-	Not Null	At least 12 years of age.	-	-	T m
membership_tier	NUMBER	1	Not Null	-	1	-	T m
loyalty_points	NUMBER	-	Not Null	Above or equals to 0	0	-	T lo e

ENTITY NAME: CUSTOMERS

Attribute	Data Type	Field Size	Null / Not Null	Constraints	Default Value	References	
customer_id (PK)	NUMBER	-	Not Null	Primary key	-	-	The
member_id (FK)	VARCHAR2	6	Null	Referential constraint	-	members (member_id)	The The ass cus
type	VARCHAR2	10	Not Null	Must be either "Member" or "Guest"	-	-	The cus

ENTITY NAME: CATEGORIES

Attribute	Data Type	Field Size	Null / Not Null	Constraints	Default Value	References	
category_id (PK)	NUMBER	-	Not Null	Primary key	-	-	T e
name	VARCHAR2	30	Not Null	-	-	-	T c
description	VARCHAR2	1000	Null	-	-	-	D th

ENTITY NAME: PROMOTIONS

Attribute	Data Type	Field Size	Null / Not Null	Constraints	Default Value	References
promotion_id (PK)	NUMBER	-	Not Null	Primary key	-	-
name	VARCHAR2	30	Not Null	-	-	-
discount_type	VARCHAR2	2	Not Null	Either "%" or "RM"	-	-
discount_value	NUMBER	(6,2)	Not Null	Greater than 0	-	-
start_date	DATE	10	Not Null	-	-	-
end_date	DATE	10	Not Null	Later than valid start date	-	-

ENTITY NAME: VOUCHERS

Attribute	Data Type	Field Size	Null / Not Null	Constraints	Default Value	References	
voucher_code (PK)	VARCHAR2	15	Not Null	Primary key	-	-	The u each
name	VARCHAR2	30	Not Null	-	-	-	The vouch
description	VARCHAR2	500	-	-	-	-	The vouch
start_date	DATE	-	-	-	-	-	The s vouch
end_date	DATE	-	Not Null	Later than start_date	-	-	The e vouch
min_spend	NUMBER	4	NOT NULL	Greater than or equals to 0	-	-	The m the cu spend discou
discount_rate	NUMBER	(3, 2)	Not Null	Greater than 0 and lower than or equals to 1	-	-	The p discou subtra paid a

discount_cap	NUMBER	(6, 2)	Not Null	Greater than 0	-	-	The discount given
membership_tier	NUMBER	1	Not Null	Tier limit less than or equals to 6	1	-	The membership
redeem_point	NUMBER	-	Not Null	Must be above or equals to 0	-	-	The point redeemed

ENTITY NAME: MEMBERS_VOUCHERS

Attribute	Data Type	Field Size	Null / Not Null	Constraints	Default Value	References	
member_id (PK) (FK)	NUMBER	-	NOT NULL	Referential constraint	-	members (member_id)	T I v
voucher_code (PK) (FK)	VARCHAR2	15	NOT NULL	Referential constraint	-	vouchers (voucher_code)	T c t
leftover	NUMBER	-	NOT NULL	-	1	-	T r l

ENTITY NAME: MENUS

Attribute	Data Type	Field Size	Null / Not Null	Constraints	Default Value	References
menu_id (PK)	NUMBER	-	Not Null	Primary key	-	-
category_id (FK)	NUMBER	-	Not Null	Referential constraint	-	categories (category_id)
name	VARCHAR2	30	Not Null	-	-	-
price	NUMBER	(6, 2)	Not Null	-	-	-
description	VARCHAR2	1000	Null	-	-	-
type	VARCHAR2	10	Not Null	-	-	-

ENTITY NAME: ITEMS

attribute	Data Type	Field Size	Null / Not Null	Constraints	Default Value	References
item_id (PK)	NUMBER	-	Not Null	Primary Key	-	-
name	VARCHAR2	30	Not Null	-	-	-

ENTITY NAME: FOODS

Attribute	Data Type	Field Size	Null / Not Null	Constraints	Default Value	References
food_id (PK)	VARCHAR2	6	Not Null	Primary key	-	-
item_id (FK)	NUMBER	-	Not Null	Referential constraint	-	items (item_id)
calorie	NUMBER	-	Not Null	Greater than or equals to 0	-	-
ingredient	VARCHAR2	30	Not Null	-	-	-

ENTITY NAME: DRINKS

Attribute	Data Type	Field Size	Null / Not Null	Constraints	Default Value	References
drink_id (PK)	VARCHAR2	6	Not Null	Primary key	-	-
item_id (FK)	NUMBER	-	Not Null	Referential constraint	-	items (item_id)
calorie	NUMBER	-	Not Null	Greater than or equals to 0	-	-
available_size	VARCHAR2	30	Not Null	"Small", "Medium", "Large", "Giant".	"medium"	-

ENTITY NAME: MERCH

Attribute	Data Type	Field Size	Null / Not Null	Constraints	Default Value	References
merch_id (PK)	VARCHAR2	6	Not Null	Primary key	-	-
item_id (FK)	NUMBER	-	Not Null	Referential constraint	-	items (item_id)
type	VARCHAR2	12	Not Null	-	-	-

ENTITY NAME: STAFFS_ROLES

Attribute	Data Type	Field Size	Null / Not Null	Constraints	Default Value	References	
staff_id (PK)(FK)	VARCHAR2	6	Not Null	Referential constraint	-	staffs (staff_id)	The uni staff.
role_id (PK)(FK)	VARCHAR2	6	Not Null	Referential constraint	-	roles (role_id)	The uni role.

ENTITY NAME: BRANCHES

ATTRIBUTE	Data Type	Field Size	Null / Not Null	Constraints	Default Value	References	
branch_id (PK)(FK)	VARCHAR2	6	Not Null	Primary key	-	-	T e
manager_id (PK)(FK)	VARCHAR2	6	Not Null	Referential constraint	-	staffs (staff_id)	T s m b
address	VARCHAR2	300	Not Null	-	-	-	T b

ENTITY NAME: MENUS_IN_BRANCH

Attribute	Data Type	Field Size	Null / Not Null	Constraints	Default Value	References	
menu_id (PK) (FK)	NUMBER	-	Not Null	Referential constraint	-	menus (menu_id)	The menu
branch_id (PK) (FK)	VARCHAR2	6	Not Null	Referential constraint	-	branches (branch_id)	The branch available
stock	NUMBER	5	Null	Greater than or equals to 0	0	-	The menu in the

ENTITY NAME: MENUS_ITEMS

Attribute	Data Type	Field Size	Null / Not Null	Constraints	Default Value	References	
menu_id (PK) (FK)	NUMBER	-	Not Null	Referential constraint	-	menus (menu_id)	The menu
item_id (PK) (FK)	NUMBER	-	Not Null	Referential constraint	-	items (item_id)	The item
quantity	NUMBER	-	Not Null	Greater than or equals to 0	0	-	Quantity of item

ENTITY NAME: ORDERS

Attribute	Data Type	Field Size	Null / Not Null	Constraints	Default Value	References
order_id (PK)	NUMBER	-	Not Null	Primary key	-	-
staff_handle (FK)	VARCHAR2	6	Not Null	Referential constraint	-	staffs (staff_id)
customer_id (FK)	NUMBER	-	Not Null	Referential constraint	-	customers (customer_id)
member_id (FK)	NUMBER	-	-	Referential constraint	-	members (member_id)
branch_id (FK)	VARCHAR2	6	Not Null	Referential constraint	-	branches (branch_id)
promotion_id (FK)	NUMBER	-	Null	Referential constraint	-	promotions (promotion_id)
voucher_code (FK)	VARCHAR2	15	Null	Referential constraint	-	vouchers (voucher_code)
order_type	VARCHAR2	10	Not Null	-	-	-
total_amount	NUMBER	(6,2)	Not Null	Greater than or equals to 0	-	-
discount_amount	NUMBER	(6,2)	Null	Greater than or equals to 0	0	-

order_datetime	DATE	-	Not Null	Order_date must be today or a past date.	SYSDAT E	-	T v
status	NUMBER	1	Not Null	Between -1 and 4	0	-	T C

ENTITY NAME: ORDERS_MENU

Attribute	Data Type	Field Size	Null / Not Null	Constraints	Default Value	References	
order_id (PK) (FK)	NUMBER	-	Not Null	Referential constraint	-	orders (order_id)	The u order.
item_id (PK) (FK)	NUMBER	-	Not Null	Referential constraint	-	items (item_id)	The ite which order.
price_on_order	NUMBER	(6, 2)	Not Null	Greater than or equals to 0	-	-	The s item purcha
quantity	NUMBER	1	Not Null	Greater than or equals to 1	-	-	The n the ite the orc

ENTITY NAME: PAYMENTS

Attribute	Data Type	Field Size	Null / Not Null	Constraints	Default Value	References	
payment_id (PK)	NUMBER	-	Not Null	Primary key	-	-	The u paym
order_id (FK)	NUMBER	6	Not Null	Referential constraint	-	orders (order_id)	The c with th
amount_paid	NUMBER	(6, 2)	Not Null	Greater than or equals to 0	-	-	The to the or
payment_method	VARCHAR2	15	Not Null	-	-	-	The paym
payment_date	DATE	-	Not Null	Today or a past date.	-	-	The d made

1.5 Tables and Records

staffs

staff_id	first_name	last_name	username	password	email
1	John	Doe	jdoe	Pass@123	jdoe@example.com
2	Jane	Smith	jsmith	Pass@456	jsmith@example.com
3	Alice	Brown	abrown	Pass@789	abrown@example.com
4	Bob	White	bwhite	Pass@321	bwhite@example.com
5	Charlie	Davis	cdavis	Pass@654	cdavis@example.com
6	Daniel	Lee	dlee	Pass@987	dlee@example.com
7	Emma	Wilson	ewilson	Pass@654	ewilson@example.com
8	Michael	Anderson	manderson	Pass@321	manderson@example.com
9	Sophia	Thomas	sthenas	Pass@147	sthenas@example.com
10	Ethan	Harris	eharris	Pass@258	eharris@example.com

roles

role_id	name	description
MNG	Manager	Branches operations and staff management
CHE	Chef	Prepares food and manages kitchen staff
CAS	Cashier	Handles customer transactions and billing
WAI	Waiter	Serves food and interacts with customers

CLE	Cleaner	Maintains cleanliness of the restaurant
-----	---------	---

members

member_id	first_name	last_name	username	password	email	phone	address	city
1	Emily	Clark	EmClark01	Pass@123	eclark@example.com	0131231234	25, Jalan Anggerik 3, Taman Melati, 53100 Kuala Lumpur	Kuala Lumpur
2	Daniel	Johnson	JohnD02	Pass@456	djohnson@example.com	0142342345	10, Lorong Merpati 2, Bandar Baru Uda, 81200 Johor Bahru, Johor	Johor Bahru
3	Sophia	Miller	MillerS03	Pass@789	smiller@example.com	0153453456	88, Jalan Kenanga, Taman Indah, 75200 Melaka	Melaka
4	Michael	Wilson	WilsonM04	Pass@321	mwilson@example.com	0164564567	5, Persiaran Mawar, Seksyen 9, 40000 Shah Alam, Selangor	Shah Alam
5	Olivia	Martinez	MartinezO05	Pass@654	omartinez@example.com	0175675678	17, Jalan Rusa 4, Taman Bukit Jaya, 81300 Skudai, Johor	Skudai

mple.
com

customers

customer_id	member_id	type
1	1	Member
2	NULL	Guest
3	2	Member
4	NULL	Guest
5	3	Member

categories

category_id	name	description
1	Appetizers	Start your meal with a delicious appetizer.
2	Main Course	Delicious burgers and fried chicken meals.
3	Desserts	Sweet treats to complete your meal.
4	Beverages	Refreshing drinks to pair with your food.
5	Combo Meals	Perfectly paired meal sets with drinks and sides.
6	Merch	Branded fast food merchandise including apparel, collectibles, and accessories.

promotions

promotion_id	name	discount_type	discount_value	start_date	end_date
1	Summer Sale	%	15.00	2025-06-01	2025-06-30
2	Sunny Season Savings	RM	2.00	2025-04-01	2025-12-25
3	Festive Price Slash	%	3.00	2025-04-01	2025-05-07
4	Weekend Deal	RM	100.00	2025-04-03	2025-04-05
5	Clearance Sale	%	50.00	2025-07-01	2025-07-15

vouchers

voucher_code	name	description		start_date	end_date	min_spend	discount_rate	discount_value
VC001	Summer Promo	Discount on	summer products	26-APR-25	30-JUN-25	20	0.15	5.00
VC002	Holiday Special	Year-end holiday	discounts	26-APR-25	31-DEC-25	10	0.30	3.00
VC003	Flash Sale	Limited time flash sale	discounts	26-APR-25	07-MAY-25	50	0.10	10.00
VC004	Weekend Deal	Special discounts for	weekend shoppers	26-APR-25	09-JUN-25	20	0.25	10.00

VC005	Spring Fever	Discount on all spring menu items	26-APR- 25	31- MAY-25	20	0.18	10.00
-------	-----------------	---	---------------	---------------	----	------	-------

members_vouchers

member_id	voucher_code	leftover
1	VC001	1
2	VC002	2
3	VC003	3
4	VC004	4
5	VC005	5

menus

menu_id	category_id	name	price	description
1	1	Mozzarella Sticks	5.99	Crispy fried mozzarella sticks served with
2	1	Fries	3.99	Crispy golden fries served with ketchup.
3	1	Onion Rings	4.99	Golden crispy onion rings with dipping s

4	2	Cheeseburger	8.99	Juicy beef patty with melted cheese, lettuce
5	2	Fried Chicken	5.99	Crispy fried chicken.
6	2	Grilled Chicken Sandwich	7.99	Grilled chicken breast with lettuce and m
7	3	Chocolate Brownie	4.49	Rich chocolate brownie served with vani
8	3	Apple Pie	3.99	Warm apple pie with a flaky crust.
9	3	Milkshake	5.49	Thick and creamy milkshake available strawberry.
10	4	Soda	2.49	Refreshing carbonated drink available in
11	4	Iced Tea	2.99	Freshly brewed iced tea, sweetened or un
12	4	Coffee	1.99	Hot brewed coffee with optional cream a
13	4	Lemonade	3.49	Freshly squeezed lemonade with a hint o
14	4	Hot Chocolate	3.99	Rich and creamy hot chocolate topped w
15	5	Burger Combo	10.99	Cheeseburger with fries and a soda.
16	5	Fried Chicken Combo	12.99	2-piece fried chicken with fries and a soc

17	5	Grilled Chicken Sandwich Combo	11.49	Grilled chicken sandwich with onion rings
18	6	Logo T-shirt	19.99	Comfortable cotton t-shirt with the restaurant logo
19	6	Collector's Cup	9.99	Exclusive limited edition cup with the restaurant logo
20	6	Restaurant Hat	14.99	Stylish hat with the restaurant logo for a cool look
21	6	Keychain	4.99	Metal keychain with the restaurant brand logo
22	6	Mug	7.99	Ceramic mug featuring the restaurant logo

items

item_id	name
1	Mozzarella Sticks
2	Fries
3	Onion Rings
4	Cheeseburger
5	Fried Chicken
6	Grilled Chicken Sandwich
7	Chocolate Brownie
8	Apple Pie
9	Milkshake

10	Soda
11	Iced Tea
12	Coffee
13	Lemonade
14	Hot Chocolate
15	Burger Combo
16	Fried Chicken Combo
17	Grilled Chicken Sandwich Combo
18	Logo T-shirt
19	Collector's Cup
20	Restaurant Hat
21	Keychain
22	Mug

foods

food_id	item_id	calorie	ingredient
F00001	1	400	Mozzarella Cheese
F00002	2	400	Potatoes
F00003	3	320	Onions

F00004	4	600	Beef Patty
F00005	5	500	Chicken
F00006	6	480	Chicken Breast
F00007	7	450	Chocolate
F00008	8	350	Apple Filling
F00009	9	500	Milk and Ice Cream

drinks

drink_id	item_id	calorie	available_size
D00001	10	150	medium
D00002	11	0	large
D00003	12	5	medium
D00004	13	5	medium
D00005	14	5	medium

merch

merch_id	item_id	type	merch_id
M00001	18	apparel	m00001
M00002	19	collectible	m00002

M00003	20	apparel	m00003
M00004	21	accessories	m00004
M00005	22	drinkware	m00005

menus_items

menu_id	item_id	quantity
1	1	1
2	2	1
3	3	1
4	4	1
5	5	1
6	6	1
7	7	1
8	8	1
9	9	1
10	10	1
11	11	1
12	12	1
13	13	1

14	14	1
15	4	1
15	2	1
15	10	1
16	5	2
16	2	1
16	10	1
17	6	1
17	3	1
17	11	1

staffs_roles

staff_id	role_id
S00001	MNG
S00002	MNG
S00003	MNG
S00004	MNG
S00005	MNG
S00006	CAS

S00007	CAS
S00008	CAS
S00009	CAS
S00010	CAS

branches

branch_id	manager_id	address
B00001	S00001	123 Main St, City Center, KL
B00002	S00001	456 Elm St, Uptown, Penang
B00003	S00001	789 Oak St, Suburban Area, JB
B00004	S00001	321 Pine St, Downtown, KL
B00005	S00001	654 Maple St, Waterfront, Melaka

menus_in_branches

menu_id	branch_id	stock
1	B00001	50
2	B00001	30
3	B00001	30

4	B00001	0
5	B00001	40
1	B00002	45
2	B00002	30
3	B00002	30
4	B00002	30
6	B00002	30
1	B00003	45
2	B00003	30
3	B00003	30
4	B00003	30
5	B00003	20
10	B00003	30
15	B00003	50
16	B00003	60
1	B00004	60
2	B00004	30
3	B00004	30
4	B00004	30

5	B00004	30
1	B00005	40
2	B00005	30
3	B00005	30
4	B00005	40
5	B00005	30

orders

order_id	staff_handle	customer_id	member_id	branch_id	promotion_id	voucher_code	order_type	total_amount
1	S00006	1	1	B00001	2		Dine-In	15.97
2	S00007	2	2	B00002		VC002	Takeaway	20.97
3	S00008	3	3	B00003	3	VC003	Delivery	59.94
4	S00009	4	4	B00004			Takeaway	5.99
5	S00010	5	5	B00005			Dine-In	11.97

orders_menus

order_id	menu_id	price_on_order	quantity
1	1	5.99	2
1	2	3.99	1
2	3	4.99	1
2	6	7.99	2
3	1	5.99	2
3	15	10.99	2
3	16	12.99	2
4	1	5.99	1
5	2	3.99	3

payments

payment_id	order_id	amount_paid	payment_method	payment_date
1	1	13.63	Credit Card	26-APR-25
2	2	17.97	Cash	26-APR-25
3	3	52.15	Debit Card	26-APR-25
4	4	5.99	Mobile Payment	26-APR-25
5	5	11.97	Cash	26-APR-25

Appendix

Individual Script

Queries

1. Filter item based on category type.

```
SELECT  name AS "Item name",
        price AS "Price",
        description AS "Description",
        type AS "Category Type"
FROM menus
WHERE category_id = 1;
```

2. Identify all orders that require delivery

```
SELECT  order_id AS "Order ID",
        branch_id AS "Branch ID",
        customer_id AS "Customer ID",
        member_id AS "Member ID",
        order_datetime AS "Order Date"
FROM orders
WHERE order_type = 'Delivery';
```

Procedures

1. Update order details after customer places order

```
-- Define an object type to represent a single menu item within an order
-- Includes the menu ID, the price at the time of order, and the quantity
ordered
CREATE OR REPLACE TYPE orders_menus_type AS OBJECT (
    menu_id NUMBER,
    price_on_order NUMBER(6,2),
    quantity NUMBER
);
/

-- Multiple items per order
-- This is used to pass a list of multiple menu items for a single order
CREATE OR REPLACE TYPE orders_menus_table_type AS TABLE OF orders_menus_type;
/

-- Stored procedure
CREATE OR REPLACE PROCEDURE add_order(
    p_staff_handle IN VARCHAR2,
    p_customer_id IN NUMBER,
    p_member_id IN NUMBER,
    p_branch_id IN VARCHAR2,
```

```
p_promotion_id IN NUMBER,
p_voucher_code IN VARCHAR2,
p_order_type IN VARCHAR2,
p_total_amount IN NUMBER,
p_discount_amount IN NUMBER,
p_payment_method IN VARCHAR2,
p_amount_paid IN NUMBER,

-- Input parameter: a collection of menu items for the order
p_orders_menus IN orders_menus_table_type
)
IS
    v_order_id NUMBER;
BEGIN
    -- Insert the main order details into the orders table
    INSERT INTO orders(
        staff_handle,
        customer_id,
        member_id,
        branch_id,
        promotion_id,
        voucher_code,
        order_type,
        total_amount,
        discount_amount,
        order_datetime,
        status
    ) VALUES (
        p_staff_handle,
        p_customer_id,
        p_member_id,
        p_branch_id,
        p_promotion_id,
        p_voucher_code,
        p_order_type,
        p_total_amount,
        p_discount_amount,
        SYSDATE,
        -- Initial status
        0
    )
    -- Get the newly generated order ID
    RETURNING order_id INTO v_order_id;

    -- Loop through the collection of menu items and insert each one into the
    orders_menus table
    FOR i IN 1..p_orders_menus.COUNT LOOP
        INSERT INTO orders_menus (
```

```
        order_id,  
        menu_id,  
        price_on_order,  
        quantity  
    ) VALUES (  
        v_order_id,  
        p_orders_menus(i).menu_id,  
        p_orders_menus(i).price_on_order,  
        p_orders_menus(i).quantity  
    );  
END LOOP;  
  
COMMIT;  
EXCEPTION  
    WHEN OTHERS THEN  
        ROLLBACK;  
        RAISE;  
END;  
/
```

2. Update order status

```
CREATE OR REPLACE PROCEDURE update_status(  
    p_order_id IN NUMBER,  
    p_new_status IN NUMBER  
)  
IS  
BEGIN  
  
    IF p_new_status < -1 OR p_new_status > 4 THEN  
        DBMS_OUTPUT.PUT_LINE('Error: Invalid status value provided: ' ||  
p_new_status);  
        RAISE_APPLICATION_ERROR(-20005, 'Invalid status value provided ( ' ||  
p_new_status || '). Status must be between -1 and 4.');
```

```
    END IF;  
  
    UPDATE orders  
    SET status = p_new_status  
    WHERE order_id = p_order_id;  
  
    IF SQL%ROWCOUNT > 0 THEN  
        DBMS_OUTPUT.PUT_LINE('Order ID ' || p_order_id || ' status has been  
updated.');
```

```
    COMMIT;  
    ELSE  
        DBMS_OUTPUT.PUT_LINE('Error: Order ID ' || p_order_id || ' not  
found.');
```

```
        RAISE_APPLICATION_ERROR(-20004, 'Order ID ' || p_order_id || ' not  
found for status update.');
```

END IF;

EXCEPTION

WHEN OTHERS THEN

DBMS_OUTPUT.PUT_LINE('An error occurred while updating status of for
order ID ' || p_order_id || ' : ' || SQLERRM);

ROLLBACK;

RAISE;

END;

/

Functions

1. Calculate applicable promotion discount

```
CREATE OR REPLACE FUNCTION count_promotional_discount(  
    p_total_amount IN NUMBER,  
    p_promotion_id IN NUMBER,  
    p_discounted_value OUT NUMBER  
)  
RETURN NUMBER  
IS  
    v_discount_type VARCHAR2(2);  
    v_discount_value NUMBER(6,2);  
    v_applicable_discount NUMBER := 0;  
    v_start_date DATE;  
    v_end_date DATE;  
BEGIN  
    SELECT discount_type, discount_value, start_date, end_date  
    INTO v_discount_type, v_discount_value, v_start_date, v_end_date  
    FROM promotions  
    WHERE promotion_id = p_promotion_id;  
  
    IF NOT (v_start_date <= SYSDATE AND v_end_date >= SYSDATE) THEN  
        DBMS_OUTPUT.PUT_LINE('Promotion ID: ' || p_promotion_id || ' is not  
active on the current date.');
```

RETURN 0;

END IF;

IF v_discount_type = '%' THEN

v_applicable_discount := p_total_amount * (v_discount_value / 100.0);

ELSIF v_discount_type = 'RM' THEN

v_applicable_discount := v_discount_value;

END IF;

p_discounted_value := p_total_amount - v_applicable_discount;

RETURN v_applicable_discount;


```
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Promotional discount does not exist.');
```

RETURN 0;

```
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An error occurred while calculating discount: '
|| SQLERRM);
        RETURN 0;
END;
/
```

2. Determine if the provided payment information is valid

```
CREATE OR REPLACE FUNCTION is_payment_valid(
    final_amount IN NUMBER,
    amount_paid IN NUMBER,
    payment_method IN VARCHAR2
)
RETURN BOOLEAN
IS
BEGIN
    IF payment_method IN (
        'Credit Card',
        'Cash',
        'Debit Card',
        'Mobile Payment',
        'Gift Card'
    ) THEN
        IF amount_paid >= final_amount THEN
            DBMS_OUTPUT.PUT_LINE('Balance: ' || TO_CHAR(amount_paid -
final_amount));
            RETURN TRUE;
        ELSE
            DBMS_OUTPUT.PUT_LINE('Warning: Insufficient amount: ' ||
amount_paid);
            RETURN FALSE;
        END IF;
    ELSE
        DBMS_OUTPUT.PUT_LINE('INVALID PAYMENT METHOD: ' || payment_method);
        RETURN FALSE;
    END IF;
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An unexpected error occurred in
is_payment_valid: ' || SQLERRM);
        RAISE;
END;
/
```

Queries

1. Search item based on item name and description

```
SELECT
    m.name AS "Name",
    c.name AS "Category",
    m.description AS "Description",
    m.price AS "Price"
FROM menus m
JOIN menus_in_branches mib ON m.menu_id = mib.menu_id
JOIN branches b ON mib.branch_id = b.branch_id
JOIN categories c ON m.category_id = c.category_id
WHERE (LOWER(m.name) LIKE '%mozzarella%' OR LOWER(m.description) LIKE '%mozzarella%')
AND b.branch_id = 'B00001'
ORDER BY m.name, m.description;
```

2. Retrieve member's details, including name, membership tier and loyalty points

```
SELECT
    INITCAP(m.first_name || ' ' || m.last_name) AS "Name",
    m.membership_tier AS "Current Tier",
    m.loyalty_points AS "Loyalty Points"
FROM members m
WHERE m.member_id = 1;
```

Procedures

1. Record customer payment details

```
CREATE OR REPLACE PROCEDURE record_payment(
    p_order_id IN NUMBER,
    p_amount_paid IN NUMBER,
    p_payment_method IN VARCHAR2
) IS
    v_order_exists NUMBER;
BEGIN
    -- Check if the order exists
    SELECT COUNT(*)
    INTO v_order_exists
    FROM orders
    WHERE order_id = p_order_id;

    IF v_order_exists = 0 THEN
        DBMS_OUTPUT.PUT_LINE('Error: Order does not exist.');
```

```
-- Insert a new payment record into the 'payments' table
INSERT INTO payments (order_id, amount_paid, payment_method)
VALUES (p_order_id, p_amount_paid, INITCAP(p_payment_method));

COMMIT;

DBMS_OUTPUT.PUT_LINE('Success: Payment recorded successfully.');
```

EXCEPTION

```
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
```

/

2. Update prices of items

```
CREATE OR REPLACE PROCEDURE update_item_price(
    p_menu_id IN NUMBER,
    p_new_price IN NUMBER
) AS
    v_item_exists NUMBER;
BEGIN
    -- Check if the item exists
    SELECT COUNT(*) INTO v_item_exists
    FROM menus
    WHERE menu_id = p_menu_id;

    IF v_item_exists = 0 THEN
        DBMS_OUTPUT.PUT_LINE('Error: Menu item does not exist.');
```

RETURN;

```
    END IF;

    -- Validate the new price
    IF p_new_price <= 0 THEN
        DBMS_OUTPUT.PUT_LINE('Error: The new price must be greater than
zero.');
```

RETURN;

```
    END IF;

    -- Update the price
    UPDATE menus
    SET price = p_new_price
    WHERE menu_id = p_menu_id;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Success: Price updated successfully.');
```

```
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/

BEGIN
    update_item_price(
        p_menu_id => 1,
        p_new_price => 6.99
    );
END;
/
```

Functions

1. Calculate discount amount from applied voucher

```
CREATE OR REPLACE FUNCTION calculate_voucher_discount(
    p_voucher_code IN VARCHAR2,
    p_total_amount IN NUMBER
) RETURN NUMBER AS
    v_discount_rate NUMBER;
    v_discount_cap NUMBER;
    v_discount_amount NUMBER;
BEGIN
    -- Validate input total amount
    IF p_total_amount <= 0 THEN
        DBMS_OUTPUT.PUT_LINE('Error: The total amount must be greater than
zero. ');
        RETURN 0;
    END IF;

    -- Attempt to get voucher details
    BEGIN
        SELECT discount_rate, discount_cap
        INTO v_discount_rate, v_discount_cap
        FROM vouchers
        WHERE voucher_code = p_voucher_code;

    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            DBMS_OUTPUT.PUT_LINE('Error: Voucher code "' || p_voucher_code ||
'" not found. ');
            RETURN 0;
        WHEN OTHERS THEN
            DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
```

```
        RETURN 0;
    END;

    -- Calculate discount based on voucher
    v_discount_amount := p_total_amount * v_discount_rate;

    -- Apply discount cap if necessary
    IF v_discount_amount > v_discount_cap THEN
        v_discount_amount := v_discount_cap;
    END IF;

    DBMS_OUTPUT.PUT_LINE('Success: Discount amount of RM' ||
TO_CHAR(v_discount_amount) || ' applied using voucher "' || p_voucher_code ||
'".');

    -- Return the calculated discount amount
    RETURN v_discount_amount;
END;
/
```

2. Check member ranking based on the loyalty points they own

```
CREATE OR REPLACE FUNCTION get_member_rank(
    p_member_id IN NUMBER
) RETURN NUMBER AS
    v_rank NUMBER;
    v_loyalty_points NUMBER;
    v_count NUMBER;
BEGIN
    -- Check if member exists
    SELECT COUNT(*) INTO v_count
    FROM members
    WHERE member_id = p_member_id;

    IF v_count = 0 THEN
        DBMS_OUTPUT.PUT_LINE('Error: Member ID ' || p_member_id || ' does not
exist.');
```

```
        RETURN NULL;
    END IF;

    -- Get member's total points
    SELECT loyalty_points
    INTO v_loyalty_points
    FROM members
    WHERE member_id = p_member_id;

    -- Calculate the rank
    SELECT rank
```

UCCD2303 Group Assignment Question
February 2025 Trimester

```
    INTO v_rank
  FROM (
    SELECT member_id,
           RANK() OVER (ORDER BY loyalty_points DESC) AS rank
    FROM members
  )
  WHERE member_id = p_member_id;

  DBMS_OUTPUT.PUT_LINE('Success: Member Rank: ' || v_rank || ', Total
Points: ' || v_loyalty_points);

  RETURN v_rank;

EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
    RETURN NULL;
END;
```

Queries

1. Retrieve specified customers' order history

```
SELECT
    O.order_id AS "Order no.",
    O.order_datetime AS "Order Date",
    O.order_type AS "Order Mode",
    LISTAGG(Tmp.menu_item, CHR(10))
        WITHIN GROUP (ORDER BY Tmp.menu_id) AS "Menu Item(s) with Quantity",
    SUM(Tmp.total_price) AS "Total Item(s) Price (RM)",
    (O.total_amount - O.discount_amount) AS "Final Amount (RM)",
    O.status AS "Status",
    O.branch_id AS "Branch",
    B.address AS "Branch Location",
    O.staff_handle AS "Staff"
FROM orders O, branches B,
(
    SELECT
        OM.order_id,
        M.menu_id,
        M.menu_id || ' ' || M.name || ' (Qty: ' || SUM(OM.quantity) || ')'
    AS menu_item,
        SUM(OM.price_on_order * OM.quantity) AS total_price
    FROM orders_menus OM, menus M
    WHERE OM.menu_id = M.menu_id
    GROUP BY OM.order_id, M.menu_id, M.name
) Tmp
WHERE O.order_id = Tmp.order_id
AND O.branch_id = B.branch_id
AND O.customer_id = 1
GROUP BY O.order_id, O.order_datetime, O.order_type,
    O.total_amount, O.discount_amount,
    O.status, O.branch_id, B.address, O.staff_handle
ORDER BY O.order_datetime DESC, O.order_id;
```

2. Identify best-selling items

```
SELECT OM.menu_id AS "Menu ID",
    M.name AS "Menu Item",
    SUM(OM.quantity) AS "Total Quantity Ordered"
FROM orders_menus OM, menus M
WHERE OM.menu_id = M.menu_id
GROUP BY OM.menu_id, M.name
HAVING SUM(OM.quantity) = (
    SELECT MAX(total_quantity)
    FROM (
```

```
        SELECT SUM(OM.quantity) AS total_quantity
        FROM orders_menus OM, menus M
        WHERE OM.menu_id = M.menu_id
        GROUP BY M.name
    )
)
ORDER BY OM.menu_id;
```

Procedures

1. Update the stock level of specific branch

```
CREATE OR REPLACE PROCEDURE update_stock_level(
    b_branch_id IN VARCHAR2,
    m_menu_id IN NUMBER,
    v_stock_number IN NUMBER
)
IS
    branch_menu_exist NUMBER;
BEGIN
    -- Check branch ID and menu ID exist or not
    SELECT COUNT(*) INTO branch_menu_exist
    FROM menus_in_branches
    WHERE branch_id = b_branch_id
    AND menu_id = m_menu_id;

    IF branch_menu_exist > 0 THEN
        IF v_stock_number >= 0 THEN
            UPDATE menus_in_branches
            SET stock = v_stock_number
            WHERE branch_id = b_branch_id
            AND menu_id = m_menu_id;
            DBMS_OUTPUT.PUT_LINE('Menu ID ' || m_menu_id || ' in Branch ID '
|| b_branch_id || ' has update stock level to ' || v_stock_number || '.');
        ELSE
            DBMS_OUTPUT.PUT_LINE('Error: The new stock number must be larger
than or equal to zero. ');
            RETURN;
        END IF;
    ELSE
        RAISE NO_DATA_FOUND;
    END IF;

    COMMIT;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
```



```
        DBMS_OUTPUT.PUT_LINE('Error: Menu ID ' || m_menu_id || ' in Branch ID ' || b_branch_id || ' is not found.');
```

```
    WHEN OTHERS THEN
```

```
        DBMS_OUTPUT.PUT_LINE('An error occurred when updating the stock level of Menu ID ' || m_menu_id || ' for branch ID ' || b_branch_id || ' : ' || SQLERRM);
```

```
END;
```

```
/
```

2. Show end of day summary of each branch based on the date

```
CREATE OR REPLACE PROCEDURE end_of_day_summary(  
    date_input IN VARCHAR2  
)  
IS
```

```
    NO_ORDERS EXCEPTION;  
    ERROR_DATE EXCEPTION;
```

```
    cur_date DATE;  
    has_data BOOLEAN := FALSE;
```

```
    -- Cursor for sales summary  
    CURSOR branch_cursor (cur_date_param DATE) IS  
        SELECT O.branch_id, B.address,  
               SUM(O.total_amount - O.discount_amount) AS total_order_value,  
               COUNT(O.order_id) AS number_of_orders  
        FROM orders O, branches B  
        WHERE O.branch_id = B.branch_id  
        AND TRUNC(O.order_datetime) = cur_date_param  
        GROUP BY O.branch_id, B.address;
```

```
    -- Cursor for items sold  
    CURSOR item_cursor (branch_id_param VARCHAR2, cur_date_param DATE) IS  
        SELECT OM.menu_id, M.name, SUM(OM.quantity) AS qty  
        FROM orders_menus OM, menus M, orders O  
        WHERE O.branch_id = branch_id_param  
        AND OM.menu_id = M.menu_id  
        AND O.order_id = OM.order_id  
        AND TRUNC(O.order_datetime) = cur_date_param  
        GROUP BY OM.menu_id, M.name  
        ORDER BY OM.menu_id;
```

```
    -- Cursor for staff  
    CURSOR staff_cursor (branch_id_param VARCHAR2, cur_date_param DATE) IS  
        SELECT DISTINCT O.staff_handle, S.first_name || ' ' || S.last_name AS  
full_name  
        FROM orders O, staffs S
```

```
WHERE O.branch_id = branch_id_param
AND O.staff_handle = S.staff_id
AND TRUNC(O.order_datetime) = cur_date_param
ORDER BY O.staff_handle;

BEGIN
    -- Validate date format
    IF NOT REGEXP_LIKE(date_input, '^\\d{2}-\\d{2}-\\d{4}$') THEN
        RAISE ERROR_DATE;
    END IF;

    -- Convert date format
    BEGIN
        cur_date := TO_DATE(date_input, 'DD-MM-YYYY');
    EXCEPTION
        WHEN OTHERS THEN
            RAISE ERROR_DATE;
    END;

    FOR branch_rec IN branch_cursor(cur_date) LOOP

        has_data := TRUE;

        -- Display branch info
        DBMS_OUTPUT.PUT_LINE(RPAD('Branch ID', 15) ||
                               RPAD('Location', 35) ||
                               RPAD('Total Orders Amount', 25) ||
                               RPAD('Number of Orders', 16));

        DBMS_OUTPUT.PUT_LINE(RPAD(branch_rec.branch_id, 15) ||
                               RPAD(branch_rec.address, 35) ||
                               RPAD('RM' || branch_rec.total_order_value, 25) ||
                               LPAD(branch_rec.number_of_orders, 16) ||
CHR(10));

        -- Display staff info
        DBMS_OUTPUT.PUT_LINE('----- Staffs -----');
        DBMS_OUTPUT.PUT_LINE(RPAD('Staff ID', 15) || RPAD('Staff Name', 30));

        FOR staff_rec IN staff_cursor(branch_rec.branch_id, cur_date) LOOP
            DBMS_OUTPUT.PUT_LINE(RPAD(staff_rec.staff_handle, 15) ||
                                   RPAD(staff_rec.full_name, 30) || CHR(10));
        END LOOP;

        -- Display items sold
        DBMS_OUTPUT.PUT_LINE('----- Items Sold -----');
        DBMS_OUTPUT.PUT_LINE(RPAD('Menu ID', 15) ||
```

```
        RPAD('Item Name', 30) ||  
        RPAD('Quantity Sold', 13));  
  
    FOR item_rec IN item_cursor(branch_rec.branch_id, cur_date) LOOP  
        DBMS_OUTPUT.PUT_LINE(RPAD(item_rec.menu_id, 15) ||  
                               RPAD(item_rec.name, 30) ||  
                               LPAD(item_rec.qty, 13));  
    END LOOP;  
  
    DBMS_OUTPUT.PUT_LINE(CHR(10));  
END LOOP;  
  
IF NOT has_data THEN  
    RAISE NO_ORDERS;  
END IF;  
  
EXCEPTION  
    WHEN ERROR_DATE THEN  
        DBMS_OUTPUT.PUT_LINE('Error: Invalid date format (Please use DD-MM-  
YYYY) or the date provided is not exist.');
```

```
    WHEN NO_ORDERS THEN  
        DBMS_OUTPUT.PUT_LINE('Error: No orders data found for the specified  
date.');
```

```
    WHEN OTHERS THEN  
        DBMS_OUTPUT.PUT_LINE('An error occurred when making end of day  
summary: ' || SQLERRM);  
END;  
/
```

Functions

1. Verify if voucher applied by member is valid

```
CREATE OR REPLACE FUNCTION verify_voucher(  
    v_voucher_code IN VARCHAR2,  
    m_member_id IN NUMBER  
)  
RETURN NUMBER  
IS  
    cur_voucher_leftover NUMBER;  
    cur_start_date DATE;  
    cur_end_date DATE;  
BEGIN  
    SELECT leftover  
    INTO cur_voucher_leftover  
    FROM members_vouchers  
    WHERE member_id = m_member_id  
    AND voucher_code = v_voucher_code;
```

```
SELECT start_date, end_date
INTO cur_start_date, cur_end_date
FROM vouchers
WHERE voucher_code = v_voucher_code;

-- Check if user don't have voucher
IF cur_voucher_leftover <= 0 THEN
    DBMS_OUTPUT.PUT_LINE('There is no leftover voucher for voucher code ' || v_voucher_code);
    RETURN 0;

-- Check voucher start date
ELSIF SYSDATE >= cur_start_date AND SYSDATE <= cur_end_date THEN
    DBMS_OUTPUT.PUT_LINE('Voucher code ' || v_voucher_code || ' is valid. ');
    RETURN 1;

ELSE
    DBMS_OUTPUT.PUT_LINE('Voucher code ' || v_voucher_code || ' is invalid');
    RETURN 0;
END IF;

EXCEPTION
WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Error: Voucher Code ' || v_voucher_code || ' or Member ID ' || m_member_id || ' are not found. ');
    RETURN 0;
WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('An error occurred when identifying the validation of voucher for voucher code ' || v_voucher_code || ' : ' || SQLERRM);
    RETURN 0;
END;
/
```

2. Calculate average order price within specified date range

```
CREATE OR REPLACE FUNCTION avg_order_amount(
    avg_start_date IN VARCHAR2,
    avg_end_date IN VARCHAR2
)
RETURN NUMBER
IS
    cur_avg_order_amount NUMBER(5,2);
    start_date DATE;
    end_date DATE;
    ERROR_DATE EXCEPTION;
```

UCCD2303 Group Assignment Question
February 2025 Trimester

```
BEGIN

    -- Validate date format
    IF NOT REGEXP_LIKE(avg_start_date, '^d{2}-d{2}-d{4}$') OR
    NOT REGEXP_LIKE(avg_end_date, '^d{2}-d{2}-d{4}$') THEN
        RAISE ERROR_DATE;
    END IF;

    -- Convert date format
    BEGIN
        start_date := TO_DATE(avg_start_date, 'DD-MM-YYYY');
        end_date := TO_DATE(avg_end_date, 'DD-MM-YYYY');
    EXCEPTION
        WHEN OTHERS THEN
            RAISE ERROR_DATE;
    END;

    SELECT AVG(total_amount - discount_amount)
    INTO cur_avg_order_amount
    FROM orders
    WHERE order_datetime BETWEEN start_date AND end_date;

    -- Check if no order withn the date range
    IF cur_avg_order_amount IS NULL THEN
        DBMS_OUTPUT.PUT_LINE('Error: No orders found within the specified date
range. ');
        RETURN 0;

    ELSE
        RETURN cur_avg_order_amount;

    END IF;

EXCEPTION
    WHEN ERROR_DATE THEN
        DBMS_OUTPUT.PUT_LINE('Error: Invalid date format (Please use DD-MM-
YYYY) or the date provided is not exist. ');
        RETURN 0;
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An error occurred when calculate the average
order amount within specified date range: ' || SQLERRM);
        RETURN 0;
END;
/
```

Queries

1. Retrieve all items available on the menu for a branch.

```
SELECT
    M.name AS "Item",
    M.description AS "Description",
    C.name AS "Category",
    M.price AS "Price (RM)"
FROM menus_in_branches MB, menus M, categories C
WHERE MB.menu_id = M.menu_id
AND M.category_id = C.category_id
AND MB.branch_id = 'B' || LPAD('&branch_id', 5, '0');
```

2. Display receipt with transaction details based on payment_id.

```
SELECT
    P.order_id,
    TO_CHAR(P.payment_datetime, 'DD-MON-YYYY HH24:MI:SS') AS "Payment
Datetime",
    P.payment_method AS "Payment Method",
    O.staff_handle AS "Staff Handling",
    LISTAGG( TO_CHAR(M.menu_id, '000') || ' - ' || M.name || ' (' ||
OM.quantity || ' x ' || ROUND(OM.price_on_order, 2) || ')', CHR(10))
        WITHIN GROUP (ORDER BY M.menu_id) AS " ID - Items (Quantity x Unit
Price)",
    TO_CHAR(O.total_amount, 'FM9990.00') AS "Total Amt. (RM)",
    TO_CHAR(O.discount_amount, 'FM9990.00') AS "Discount (RM)",
    TO_CHAR((O.total_amount - O.discount_amount), 'FM9990.00') AS "Final Amt.
(RM)"
FROM payments P
JOIN orders O ON P.order_id = O.order_id
JOIN orders_menus OM ON O.order_id = OM.order_id
JOIN menus M ON OM.menu_id = M.menu_id
WHERE p.payment_id = '&payment_id'
GROUP BY P.order_id, P.payment_datetime, P.payment_method, O.staff_handle,
O.total_amount, O.discount_amount;
```

Procedures

1. Add or remove an item in a specific branch.

```
CREATE OR REPLACE PROCEDURE add_delete_branch_item (  
    b_branch_id IN VARCHAR2,  
    b_menu_id IN NUMBER,  
    status_code OUT NUMBER  
)  
IS  
    v_exist NUMBER;  
BEGIN  
    -- If record of given branch and menu not exist, throws "NO_DATA_FOUND"  
    SELECT menu_id INTO v_exist  
    FROM menus_in_branches  
    WHERE branch_id = b_branch_id  
    AND menu_id = b_menu_id;  
  
    -- If found, delete the menu from the branch  
    DELETE FROM menus_in_branches  
    WHERE menu_id = b_menu_id  
    AND branch_id = b_branch_id;  
  
    COMMIT;  
    DBMS_OUTPUT.PUT_LINE('Deleted item (menu_id = ' || b_menu_id || ') from  
branch (branch_id = ' || b_branch_id || ')');  
    status_code := 0;  
  
EXCEPTION  
    -- If menu not exist, add in the menu  
    WHEN NO_DATA_FOUND THEN  
        BEGIN  
            INSERT INTO menus_in_branches(menu_id, branch_id)  
            VALUES (b_menu_id, b_branch_id);  
  
            COMMIT;  
            DBMS_OUTPUT.PUT_LINE('Inserted item (menu_id = ' || b_menu_id ||  
) to branch (branch_id = ' || b_branch_id || ')');  
            status_code := 0; -- Indicate Success  
        EXCEPTION  
            WHEN OTHERS THEN  
                DBMS_OUTPUT.PUT_LINE('An error occured while adding item: ' ||  
SQLERRM);  
                status_code := -1; -- Indiacte Failed, due to menu_id not  
existed  
        END;  
    WHEN OTHERS THEN  
        DBMS_OUTPUT.PUT_LINE('An error occurred while deleting item: ' ||  
SQLERRM);
```

```
        status_code := -1; -- Indicate Failed  
END;  
/
```

2. Assess member eligibility for rewards and execute redemption transactions.

```
CREATE OR REPLACE PROCEDURE redeem_reward(  
    m_member_id IN NUMBER,  
    v_voucher_code IN VARCHAR2,  
    status_code OUT NUMBER  
)  
IS  
    hold_member_point NUMBER;  
    hold_redeem_point NUMBER;  
    hold_voucher_leftover NUMBER;  
    insufficient_point EXCEPTION;  
BEGIN  
    SELECT loyalty_points  
    INTO hold_member_point  
    FROM members  
    WHERE member_id = m_member_id;  
  
    SELECT redeem_point  
    INTO hold_redeem_point  
    FROM vouchers  
    WHERE voucher_code = v_voucher_code;  
  
    -- If member have insufficient point, throw insufficient_point EXCEPTION  
    IF hold_member_point < hold_redeem_point THEN  
        RAISE insufficient_point;  
    END IF;  
  
    -- Deduct User Loyalty Points  
    UPDATE members  
    SET loyalty_points = hold_member_point - hold_redeem_point  
    WHERE member_id = m_member_id;  
  
    BEGIN  
        -- Get member current voucher leftover.  
        -- Exception will occur if member doesn't own the voucher before.  
=> Create new record  
        SELECT leftover  
        INTO hold_voucher_leftover  
        FROM members_vouchers  
        WHERE member_id = m_member_id  
        AND voucher_code = v_voucher_code;  
  
        -- Add 1 voucher usage to member.
```



```
        UPDATE members_vouchers
        SET leftover = hold_voucher_leftover + 1
        WHERE member_id = m_member_id
        AND voucher_code = v_voucher_code;

    EXCEPTION
        -- Insert a new voucher record for member (Default leftover is 1)
        WHEN NO_DATA_FOUND THEN
            INSERT INTO members_vouchers (member_id, voucher_code)
            VALUES (m_member_id, v_voucher_code);
            COMMIT;
    END;

    COMMIT;
    -- Success
    DBMS_OUTPUT.PUT_LINE('Reward redeemed successfully!');
    status_code := 0;
EXCEPTION
    WHEN insufficient_point THEN
        DBMS_OUTPUT.PUT_LINE('Insufficient Point!');
        status_code := -1;
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Error: Invalid voucher selection!');
        status_code := -1;
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An error occurred while redeeming reward: ' ||
SQLERRM);
        status_code := -1;
END;
/
```

Functions

1. Determine if an item is available in a specific branch and is in stock.

```
CREATE OR REPLACE FUNCTION branch_item_available (  
    b_branch_id IN VARCHAR2,  
    m_menu_id IN NUMBER  
)  
RETURN NUMBER  
IS  
    hold_stock NUMBER;  
BEGIN  
    -- Checking for item availability, and get stock if exist.  
    SELECT stock INTO hold_stock  
    FROM menus_in_branches  
    WHERE branch_id = b_branch_id  
    AND menu_id = m_menu_id;  
  
    IF hold_stock = 0 THEN  
        DBMS_OUTPUT.PUT_LINE('Item is out of stock.');        RETURN 0;  
    ELSE  
        DBMS_OUTPUT.PUT_LINE('Item available with stock: ' || hold_stock);  
        RETURN hold_stock;  
    END IF;  
EXCEPTION  
    WHEN NO_DATA_FOUND THEN  
        DBMS_OUTPUT.PUT_LINE('Menu item does not exist in branch.');        RETURN -1;  
    WHEN OTHERS THEN  
        DBMS_OUTPUT.PUT_LINE('An error occurred while checking item  
availability: ' || SQLERRM);  
        RETURN -1;  
END;  
/
```

2. Calculate the loyalty points gained from an order and add the point to respective member's loyalty point.

```
CREATE OR REPLACE FUNCTION update_user_loyalty_point (  
    o_order_id IN NUMBER  
)  
RETURN NUMBER  
IS  
    hold_member_id NUMBER;  
    hold_user_point NUMBER;  
    new_point NUMBER;  
BEGIN  
    -- Retrieve the member_id and the final amount for the order.
```

```
BEGIN
    SELECT member_id, FLOOR(total_amount - discount_amount) * 3
    INTO hold_member_id, new_point
    FROM orders WHERE order_id = o_order_id;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Error: Order not exist. ');
        RETURN -1;
END;

-- Get user's current loyalty points
BEGIN
    SELECT loyalty_points
    INTO hold_user_point
    FROM members WHERE member_id = hold_member_id;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Error: Member not exist. ');
        RETURN -1;
END;

-- Calculate updated loyalty points (current member point + order price *
3)
new_point := hold_user_point + new_point;

-- Update the user's loyalty points
UPDATE members
SET loyalty_points = new_point
WHERE member_id = hold_member_id;
COMMIT;
DBMS_OUTPUT.PUT_LINE('Point successfully added to member! ');
-- Return updated loyalty points for system usage
RETURN new_point;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Error: Unable to find order/member. ');
        RETURN -1;
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An error occurred while adding loyalty point to
user: ' || SQLERRM);
        RETURN -1;
END;
/
```

<End of Individual Script>

Driver Code

Procedure

1. Update order details after customer places order

```
DECLARE
    order_details orders_menus_table_type := orders_menus_table_type();

    v_staff_handle VARCHAR2(6) := 'S00001';
    v_customer_id NUMBER := 6;
    v_member_id NUMBER := 1;
    v_branch_id VARCHAR2(6) := 'B00001';
    v_order_type VARCHAR2(20) := 'Dine-In';
    v_discount_amount NUMBER := 0;
    v_amount_paid NUMBER := 35.00;
    v_payment_method VARCHAR2(50) := 'Cash';

    v_promotion_id NUMBER := NULL;
    v_voucher_code VARCHAR(15) := NULL;

    v_total_amount NUMBER(6, 2) := 0;
    v_final_amount NUMBER(6, 2);
    v_promotion_discount NUMBER(6, 2) := 0;
    v_voucher_discount NUMBER(6, 2) := 0;
BEGIN
    order_details.EXTEND;
    order_details(order_details.COUNT) := orders_menus_type(menu_id => 1,
price_on_order => 11.98, quantity => 2);

    order_details.EXTEND;
    order_details(order_details.COUNT) := orders_menus_type(menu_id => 2,
price_on_order => 3.99, quantity => 1);

    order_details.EXTEND;
    order_details(order_details.COUNT) := orders_menus_type(menu_id => 16,
price_on_order => 12.99, quantity => 1);

    FOR i IN 1..order_details.COUNT LOOP
        v_total_amount := v_total_amount + (order_details(i).price_on_order *
order_details(i).quantity);
    END LOOP;

    IF v_promotion_id IS NOT NULL THEN
        v_promotion_discount := count_promotional_discount(v_total_amount,
v_promotion_id, v_final_amount);
        v_discount_amount := v_discount_amount + v_promotion_discount;
    END IF;
```

```
    IF v_voucher_code IS NOT NULL THEN
        v_voucher_discount := calculate_voucher_discount(v_voucher_code,
v_total_amount);
        v_discount_amount := v_discount_amount + v_voucher_discount;
    END IF;

    v_final_amount := v_total_amount - v_discount_amount;

    add_order(
        p_staff_handle => v_staff_handle,
        p_customer_id => v_customer_id,
        p_member_id => v_member_id,
        p_branch_id => v_branch_id,
        p_promotion_id => v_promotion_id,
        p_voucher_code => v_voucher_code,
        p_order_type => v_order_type,
        p_total_amount => v_total_amount,
        p_discount_amount => v_discount_amount,
        p_payment_method => v_payment_method,
        p_amount_paid => v_amount_paid,
        p_orders_menus => order_details
    );
    DBMS_OUTPUT.PUT_LINE('Order recorded successfully. Total Amount: ' ||
v_total_amount || ', Discount: ' || v_discount_amount || ', Final Amount: ' ||
v_final_amount);
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/
```

2. Determine if the provided payment information is valid

```
-- Valid status
DECLARE
    v_order_id NUMBER := 2;
    v_new_status NUMBER := 4;
BEGIN
    DBMS_OUTPUT.PUT_LINE('Attempting to update order ' || v_order_id || ' to
status ' || v_new_status);
    update_status(p_order_id => v_order_id, p_new_status => v_new_status);
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Caught error in calling block: ' || SQLERRM);
END;
/

-- Invalid status
DECLARE
    v_order_id NUMBER := 2;
    v_new_status NUMBER := 10;
BEGIN
    DBMS_OUTPUT.PUT_LINE('Attempting to update order ' || v_order_id || ' to
status ' || v_new_status);
    update_status(p_order_id => v_order_id, p_new_status => v_new_status);
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Caught error in calling block: ' || SQLERRM);
END;
/

-- Invalid order
DECLARE
    v_order_id NUMBER := 10;
    v_new_status NUMBER := 4;
BEGIN
    DBMS_OUTPUT.PUT_LINE('Attempting to update order ' || v_order_id || ' to
status ' || v_new_status);
    update_status(p_order_id => v_order_id, p_new_status => v_new_status);
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Caught error in calling block: ' || SQLERRM);
END;
/
```

Function

1. Calculate applicable promotion discount

```
DECLARE
    v_total_amount NUMBER := 50;
    v_promotion_id NUMBER := 2;
    v_promotion_discount NUMBER(6, 2) := 0;
    v_discounted_amount NUMBER(6,2) := 0;
BEGIN
    v_promotion_discount := count_promotional_discount(v_total_amount,
v_promotion_id, v_discounted_amount);
    IF v_promotion_discount = 0 THEN
        DBMS_OUTPUT.PUT_LINE('No applicable discount for Promotion ID: ' ||
v_promotion_id);
    ELSE
        DBMS_OUTPUT.PUT_LINE('Promotion ID: ' || v_promotion_id);
        DBMS_OUTPUT.PUT_LINE('Total amount: ' || v_total_amount);
        DBMS_OUTPUT.PUT_LINE('Discounted amount: ' || v_promotion_discount);
        DBMS_OUTPUT.PUT_LINE('Final amount: ' || v_discounted_amount);
    END IF;
END;
/
```

2. Determine if the provided payment information is valid.

```
-- Valid payment
DECLARE
    v_total_amount NUMBER := 50;
    v_paid_amount NUMBER := 50;
    -- Assign a value to v_payment_method for testing
    v_payment_method VARCHAR2(15) := 'Cash';
    v_valid_payment BOOLEAN := FALSE;
BEGIN
    v_valid_payment := is_payment_valid(v_total_amount, v_paid_amount,
v_payment_method);
    IF v_valid_payment = TRUE THEN
        DBMS_OUTPUT.PUT_LINE('Valid Payment');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Invalid Payment. Try again');
    END IF;
END;
/

-- Invalid payment method
DECLARE
    v_total_amount NUMBER := 50;
    v_paid_amount NUMBER := 50;
    -- Assign a value to v_payment_method for testing
```

UCCD2303 Group Assignment Question
February 2025 Trimester

```
v_payment_method VARCHAR2(15) := '-';
v_valid_payment BOOLEAN := FALSE;
BEGIN
    v_valid_payment := is_payment_valid(v_total_amount, v_paid_amount,
v_payment_method);
    IF v_valid_payment = TRUE THEN
        DBMS_OUTPUT.PUT_LINE('Valid Payment');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Invalid Payment. Try again');
    END IF;
END;
/

-- Invalid payment amount
DECLARE
    v_total_amount NUMBER := 50;
    v_paid_amount NUMBER := 40;
    -- Assign a value to v_payment_method for testing
    v_payment_method VARCHAR2(15) := 'Cash';
    v_valid_payment BOOLEAN := FALSE;
BEGIN
    v_valid_payment := is_payment_valid(v_total_amount, v_paid_amount,
v_payment_method);
    IF v_valid_payment = TRUE THEN
        DBMS_OUTPUT.PUT_LINE('Valid Payment');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Invalid Payment. Try again');
    END IF;
END;
/
```


Procedure

1. Record Customer Payment Details

```
-- Valid test case
BEGIN
    record_payment(
        p_order_id => 1,
        p_amount_paid => 13.63,
        p_payment_method => 'Credit Card'
    );
END;
/

-- Invalid test case (when the order ID of 100 doesn't exist in orders table)
BEGIN
    record_payment(
        p_order_id => 100,
        p_amount_paid => 13.63,
        p_payment_method => 'Credit Card'
    );
END;
/
```

2. Update Prices of Items

```
-- Valid test case
BEGIN
    update_item_price(
        p_menu_id => 1,
        p_new_price => 6.99
    );
END;
/

-- Invalid test case (when the menu ID of 100 doesn't exist in menus table)
BEGIN
    update_item_price(
        p_menu_id => 100,
        p_new_price => 6.99
    );
END;
/

-- Invalid test case (when the new price is equal or less than 0)
BEGIN
    update_item_price(
```

```
        p_menu_id => 1,  
        p_new_price => 0  
    );  
END;  
/
```

Function

1. Calculate Discount Amount from Applied Voucher

```
-- Valid test case  
SELECT calculate_voucher_discount(  
    p_voucher_code => 'VC002',  
    p_total_amount => 100  
    ) AS discount  
FROM dual;  
  
-- Invalid test case (when the voucher code of VC100 doesn't exist in vouchers  
table)  
SELECT calculate_voucher_discount(  
    p_voucher_code => 'VC100',  
    p_total_amount => 100  
    ) AS discount  
FROM dual;  
  
-- Invalid test case (when the total amount is equal or less than 0)  
SELECT calculate_voucher_discount(  
    p_voucher_code => 'VC002',  
    p_total_amount => 0  
    ) AS discount  
FROM dual;
```

2. Check member ranking based on the loyalty points the own

```
-- Valid test case  
SELECT get_member_rank(5) FROM dual;  
  
-- Invalid test case (when the member id of 100 doesn't exist in members  
table)  
SELECT get_member_rank(100) FROM dual;
```

Procedure

1. Update the stock level of specific branch

```
-- valid test cases
EXEC update_stock_level ('B00001',4,50);

-- invalid test case (Stock number cant be negative)
EXEC update_stock_level ('B00001',1,-10);

-- invalid test case (Invalid branch ID)
EXEC update_stock_level ('B00006',1,-10);
```

2. Show end of day summary of each branch based on the date

```
-- valid test cases
EXEC end_of_day_summary ('27-04-2025');

-- invalid test case (Invalid date provided)
EXEC end_of_day_summary ('31-02-2025');

-- invalid test case (Invalid date format)
EXEC end_of_day_summary ('2025-04-27');

-- invalid test case (No order data on that date)
EXEC end_of_day_summary ('30-12-2025');
```

Function

1. Verify if voucher applied by member is valid

```
-- valid test case
DECLARE
    v_voucher_code VARCHAR2(50) := 'VC001';
    v_member_id NUMBER := 1;
    v_result NUMBER;
BEGIN
    v_result := verify_voucher(v_voucher_code, v_member_id);

    IF v_result = 0 THEN
        DBMS_OUTPUT.PUT_LINE('Error: Voucher Code ' || v_voucher_code || ' is not
valid for Member ID ' || v_member_id || ', so it is unable to use.');
```

```
        ELSE
            DBMS_OUTPUT.PUT_LINE('Voucher Code ' || v_voucher_code || ' is valid for
Member ID ' || v_member_id || ' to use.');
```

```
        END IF;
END;
```

```
/
-- invalid test case
DECLARE
  v_voucher_code VARCHAR2(50) := 'VC005';
  v_member_id NUMBER := 1;
  v_result NUMBER;
BEGIN
  v_result := verify_voucher(v_voucher_code, v_member_id);

  IF v_result = 0 THEN
    DBMS_OUTPUT.PUT_LINE('Error: Voucher Code ' || v_voucher_code || ' is not
valid for Member ID ' || v_member_id || ', so it is unable to use.');
```

2. Calculate average order price within specified date range

```
-- invalid test case (no order data)
DECLARE
  v_start_date VARCHAR2(10) := '01-01-2024';
  v_end_date VARCHAR2(10) := '31-12-2024';
  v_avg_amount NUMBER(5,2);
BEGIN
  v_avg_amount := avg_order_amount(v_start_date, v_end_date);

  IF v_avg_amount > 0 THEN
    DBMS_OUTPUT.PUT_LINE('Average order amount from ' || v_start_date || '
to ' || v_end_date || ' is RM ' || v_avg_amount);
  ELSE
    DBMS_OUTPUT.PUT_LINE('Error: No average amount calculated due to error
or no data.');
```

UCCD2303 Group Assignment Question
February 2025 Trimester

```
v_avg_amount := avg_order_amount(v_start_date, v_end_date);

IF v_avg_amount > 0 THEN
    DBMS_OUTPUT.PUT_LINE('Average order amount from ' || v_start_date || '
to ' || v_end_date || ' is RM ' || v_avg_amount);
ELSE
    DBMS_OUTPUT.PUT_LINE('Error: No average amount calculated due to error
or no data.');
```

END IF;

END;

/

-- valid test cases

DECLARE

 v_start_date VARCHAR2(10) := '01-01-2024';

 v_end_date VARCHAR2(10) := '30-06-2025';

 v_avg_amount NUMBER(5,2);

BEGIN

 v_avg_amount := avg_order_amount(v_start_date, v_end_date);

 IF v_avg_amount > 0 THEN

 DBMS_OUTPUT.PUT_LINE('Average order amount from ' || v_start_date || '
to ' || v_end_date || ' is RM ' || v_avg_amount);

 ELSE

 DBMS_OUTPUT.PUT_LINE('Error: No average amount calculated due to error
or no data.');

 END IF;

END;

/

Procedure

1. Add or remove item menu in a specific branch.

```
----- Validation Query Below -----  
SELECT M.menu_id, name  
FROM menus_in_branches MB, menus M  
WHERE MB.menu_id = M.menu_id  
AND branch_id = 'B00001';  
----- Validation Query End -----  
  
-- Valid case (Adding menu_id=15 into branch_id=B00001)  
DECLARE  
    -- Variable to receive the status code (0 mean success, -1 mean failed)  
    status_code NUMBER;  
BEGIN  
    add_delete_branch_item('B00001', 15, status_code);  
    DBMS_OUTPUT.PUT_LINE('Status Code: ' || status_code);  
END;  
/  
  
-- Valid case (Removing menu_id=15 from branch_id=B00001)  
DECLARE  
    -- Variable to receive the status code (0 mean success, -1 mean failed)  
    status_code NUMBER;  
BEGIN  
    add_delete_branch_item('B00001', 15, status_code);  
    DBMS_OUTPUT.PUT_LINE('Status Code: ' || status_code);  
END;  
/  
  
-- Invalid case (Adding non-existing menu_id=99 into branch_id=B00001)  
DECLARE  
    -- Variable to receive the status code (0 mean success, -1 mean failed)  
    status_code NUMBER;  
BEGIN  
    add_delete_branch_item('B00001', 99, status_code);  
    DBMS_OUTPUT.PUT_LINE('Status Code: ' || status_code);  
END;  
/  
  
-- Invalid case (Adding menu_id=99 into non-existing branch_id=B99999)  
DECLARE  
    -- Variable to receive the status code (0 mean success, -1 mean failed)  
    status_code NUMBER;  
BEGIN  
    add_delete_branch_item('B00001', 99, status_code);  
    DBMS_OUTPUT.PUT_LINE('Status Code: ' || status_code);  
END;  
/
```

2. Assess member eligibility for a redemption reward and execute reward redemption transactions.

```
----- Validation Query Below -----  
SELECT loyalty_points, LISTAGG(voucher_code || ' (QTY: ' || leftover || ')',  
CHR(10)) WITHIN GROUP (ORDER BY voucher_code) AS "Vouchers (Quantity)"  
FROM members M  
JOIN members_vouchers MV  
ON M.member_id = MV.member_id  
WHERE M.member_id = 2  
GROUP BY loyalty_points;  
----- Validation Query End -----  
  
-- Invalid case (member_id=2 with 150 points redeem voucher "VC001" which  
require 200 points)  
DECLARE  
    -- Variable to receive the status code (0 success, -1 failed)  
    status_code NUMBER;  
BEGIN  
    redeem_reward(2, 'VC001', status_code);  
    DBMS_OUTPUT.PUT_LINE('Status Code: ' || status_code);  
END;  
/  
  
-- Valid case (member_id=2 with 150 points redeem voucher "VC003" which  
require 150 points)  
DECLARE  
    -- Variable to receive the status code (0 success, -1 failed)  
    status_code NUMBER;  
BEGIN  
    redeem_reward(2, 'VC003', status_code);  
    DBMS_OUTPUT.PUT_LINE('Status Code: ' || status_code);  
END;  
/
```

Function

1. Determine if an item is available in a specific branch and currently in stock.

```
----- Validation Query Below -----  
SELECT M.menu_id, name, stock  
FROM menus_in_branches MB, menus M  
WHERE MB.menu_id = M.menu_id  
AND branch_id = 'B00001';  
----- Validation Query End -----  
  
-- Valid case (Checking menu_id=1 in branch_id=B00001, given that the item is  
in stock)  
DECLARE  
    -- Hold stock value (-1 failed, 0 no stock, >= 1 stock quantity)  
    stock_no NUMBER;  
BEGIN  
    stock_no := branch_item_available('B00001', 1);  
    DBMS_OUTPUT.PUT_LINE('Returned Value: ' || stock_no);  
END;  
/  
  
-- Valid case (Checking menu_id=4 in branch_id=B00001, given that the item has  
no stock)  
DECLARE  
    -- Hold stock value (-1 failed, 0 no stock, >= 1 stock quantity)  
    stock_no NUMBER;  
BEGIN  
    stock_no:= branch_item_available('B00001', 4);  
    DBMS_OUTPUT.PUT_LINE('Returned Value: ' || stock_no);  
END;  
  
-- Invalid case (Checking non-existing menu_id=6 in branch_id=B00001)  
DECLARE  
    -- Hold stock value (-1 failed, 0 no stock, >= 1 stock quantity)  
    stock_no NUMBER;  
BEGIN  
    stock_no:= branch_item_available('B00001', 6);  
    DBMS_OUTPUT.PUT_LINE('Returned Value: ' || stock_no);  
END;  
/
```


2. Calculate the loyalty points gained from the order and provide updated loyalty points to the associated members.

```
----- Validation Query Below -----  
SELECT  
    order_id,  
    O.member_id AS "Belong to member_id",  
    FLOOR(total_amount - discount_amount)*3 AS "Point to Earn",  
    loyalty_points AS "Associated Member Point"  
FROM orders O, members M  
WHERE order_id = 2  
AND O.member_id = M.member_id;  
----- Validation Query End -----  
  
-- Executed by system after order completion.  
-- Assume order_id=2 is delivered to customer, the system execute the function  
-- to calculate the loyalty point earn from the order and add it to the  
-- associated member.  
-- Given that every RM1=3 points and decimals are ignored.  
DECLARE  
    -- Hold latest points (-1 failed, >= 0 latest point and success execution)  
    latest_point NUMBER;  
BEGIN  
    latest_point:= update_user_loyalty_point(2);  
    DBMS_OUTPUT.PUT_LINE('Returned Value: ' || latest_point);  
END;  
/
```

<End of Driver Code>

Script

```
/*  
COURSE CODE: UCCD2303  
PROGRAMME: CS  
GROUP NUMBER: G016  
  
Submission date and time (DD-MON-YY): 29-APR-25  
*/  
  
Rem  
Rem $Header: FoodieSQL.sql  
Rem Based on HR.sql  
REM  
Rem DESCRIPTION  
Rem      This script creates Nineteen(20) tables  
  
Rem Strong Entity   : staffs, roles, members, customers, categories,  
promotions, vouchers  
Rem Weak Entity 1st : members_vouchers, (menus <-(items)- foods, drinks,  
merch), staffs_roles, branches,  
Rem Weak Entity 2nd : menus_in_branches, menus_items, orders  
Rem Weak Entity 3rd : orders_menus, payments  
  
-----  
-----  
  
DROP SEQUENCE cur_staff_val;  
DROP SEQUENCE cur_member_val;  
DROP SEQUENCE cur_customer_val;  
DROP SEQUENCE cur_category_val;  
DROP SEQUENCE cur_promotion_val;  
DROP SEQUENCE cur_menu_val;  
DROP SEQUENCE cur_item_val;  
DROP SEQUENCE cur_food_val;  
DROP SEQUENCE cur_drink_val;  
DROP SEQUENCE cur_merch_val;  
DROP SEQUENCE cur_branch_val;  
DROP SEQUENCE cur_order_val;  
DROP SEQUENCE cur_payment_val;  
  
-----  
-----  
  
DROP TABLE staffs CASCADE CONSTRAINTS;
```

UCCD2303 Group Assignment Question
February 2025 Trimester

```
DROP TABLE roles CASCADE CONSTRAINTS;
DROP TABLE members CASCADE CONSTRAINTS;
DROP TABLE customers CASCADE CONSTRAINTS;
DROP TABLE categories CASCADE CONSTRAINTS;
DROP TABLE promotions CASCADE CONSTRAINTS;
DROP TABLE vouchers CASCADE CONSTRAINTS;
DROP TABLE members_vouchers CASCADE CONSTRAINTS;
DROP TABLE menus CASCADE CONSTRAINTS;
DROP TABLE items CASCADE CONSTRAINTS;
DROP TABLE foods CASCADE CONSTRAINTS;
DROP TABLE drinks CASCADE CONSTRAINTS;
DROP TABLE merch CASCADE CONSTRAINTS;
DROP TABLE staffs_roles CASCADE CONSTRAINTS;
DROP TABLE branches CASCADE CONSTRAINTS;
DROP TABLE menus_in_branches CASCADE CONSTRAINTS;
DROP TABLE menus_items CASCADE CONSTRAINTS;
DROP TABLE orders CASCADE CONSTRAINTS;
DROP TABLE orders_menus CASCADE CONSTRAINTS;
DROP TABLE payments CASCADE CONSTRAINTS;
```

```
-----
Prompt ~ ~ ~ Tables Creation Start ~ ~ ~
-----
```

```
-----
Rem Strong Entity : staffs, roles, members, customers, categories, promotions,
vouchers
-----
```

```
Prompt ***** # staffs # *****
Rem staffs_roles.staff_id and orders.staff_handle reference this table.
```

```
CREATE TABLE staffs (
    staff_id VARCHAR2(6) CONSTRAINT staffs_staff_id_pk PRIMARY
KEY,
    first_name VARCHAR2(20) CONSTRAINT staffs_first_name_nn NOT
NULL,
    last_name VARCHAR2(20) CONSTRAINT staffs_last_name_nn NOT
NULL,
    username VARCHAR2(20) CONSTRAINT staffs_username_nn NOT NULL,
    password VARCHAR2(20) DEFAULT 'Foodie@123456' CONSTRAINT
staffs_password_nn NOT NULL,
    email VARCHAR2(50) CONSTRAINT staffs_email_un UNIQUE,
    phone VARCHAR2(10) CONSTRAINT staffs_phone_nn NOT NULL,
```

UCCD2303 Group Assignment Question
February 2025 Trimester

```
CONSTRAINT chk_staffs_password CHECK (
    LENGTH(password) >= 8 AND
    REGEXP_LIKE(password, '[A-Z]') AND
    REGEXP_LIKE(password, '[0-9]') AND
    REGEXP_LIKE(password, '[!@#$$%^&*()?><.,;]')
),
CONSTRAINT chk_staffs_email_format CHECK (
    email IS NULL OR REGEXP_LIKE(email, '^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$')
),
CONSTRAINT chk_staffs_phone_length CHECK (
    -- Malaysian phone number validation
    (LENGTH(phone) BETWEEN 9 AND 10) AND (REGEXP_LIKE(phone, '^01[0-9]+$'))
);

-- cur_staff_val to counter latest staff ID's number
CREATE SEQUENCE cur_staff_val START WITH 1 INCREMENT BY 1;

CREATE OR REPLACE TRIGGER new_staff_id
BEFORE INSERT ON staffs
FOR EACH ROW
BEGIN
    --Assign new staff_id = 'S' + next staff ID with left padding
    :NEW.staff_id := 'S' || LPAD(cur_staff_val.NEXTVAL, 5, '0');
END;
/

Prompt *** Created staffs table ***
Prompt

-----

Prompt ***** # roles # *****
Rem staffs_roles.role_id refer this table.

CREATE TABLE roles (
    role_id VARCHAR2(6) CONSTRAINT roles_role_id_pk PRIMARY KEY,
    name VARCHAR2(20) CONSTRAINT roles_name_nn NOT
NULL,
    description
VARCHAR2(1000)
);

Prompt *** Created roles table ***
Prompt
```

```
-----
-----

Prompt ***** # members # *****
Rem customers.member_id and vouchers.member_id refer this table.

CREATE TABLE members (
    member_id NUMBER CONSTRAINT members_member_id_pk PRIMARY
KEY,
    first_name VARCHAR2(20) CONSTRAINT members_first_name_nn NOT
NULL,
    last_name VARCHAR2(20) CONSTRAINT members_last_name_nn NOT NULL,
    username VARCHAR2(20) CONSTRAINT members_username_nn NOT
NULL,
    password VARCHAR2(15) CONSTRAINT members_password_nn NOT
NULL,
    email VARCHAR2(50) CONSTRAINT members_email_un
UNIQUE,
    phone VARCHAR2(10) CONSTRAINT members_phone_nn NOT
NULL,
    address
VARCHAR2(300),
    date_of_birth DATE,
    membership_tier NUMBER(1) DEFAULT 1 CONSTRAINT members_membership_tier_nn
NOT NULL,
    loyalty_points NUMBER DEFAULT 0 CONSTRAINT members_points_nn NOT NULL,
    CONSTRAINT chk_members_password CHECK (
        LENGTH(password) >= 8 AND
        REGEXP_LIKE(password, '[A-Z]') AND
        REGEXP_LIKE(password, '[0-9]') AND
        REGEXP_LIKE(password, '[@#$$%^&*()?)><.,;]')
    ),
    CONSTRAINT chk_members_email_format CHECK (
        email IS NULL OR REGEXP_LIKE(email, '^[a-zA-Z0-9._%+-]+@[a-zA-Z0-
9.-]+\.[a-zA-Z]{2,}$')
    ),
    CONSTRAINT chk_members_phone_length CHECK (
        -- Malaysian phone number validation
        LENGTH(phone) BETWEEN 9 AND 10 AND REGEXP_LIKE(phone, '^01[0-9]+$')
    ),
    CONSTRAINT chk_members_membership_tier_limit CHECK(
        membership_tier <= 6
    )
);

CREATE OR REPLACE TRIGGER validate_DOB
BEFORE INSERT OR UPDATE ON members
```

```
FOR EACH ROW
BEGIN
    IF :NEW.date_of_birth > ADD_MONTHS(SYSDATE, -12 * 12) THEN
        RAISE_APPLICATION_ERROR(-20001, 'Date of birth must be at least 12
years ago.');
```

END IF;

```
END;
/

-- Assign next ID for new member
CREATE SEQUENCE cur_member_val START WITH 1 INCREMENT BY 1;

CREATE OR REPLACE TRIGGER new_member_id
BEFORE INSERT ON members
FOR EACH ROW
BEGIN
    :NEW.member_id := cur_member_val.NEXTVAL;
END;
/

Prompt *** Created members table ***
Prompt

-----

Prompt ***** # customers # *****
Rem orders.customer_id reference this table.
Rem This table reference members.member_id

CREATE TABLE customers (
    customer_id NUMBER CONSTRAINT customers_customer_id_pk PRIMARY
KEY,
    -- Optional since not every customer is a member
    member_id NUMBER,
    type VARCHAR2(10) CONSTRAINT customers_type_nn NOT NULL,
    CONSTRAINT customers_members_member_id_fk FOREIGN KEY (member_id)
REFERENCES members(member_id),
    CONSTRAINT chk_customers_type CHECK (type IN ('Member', 'Guest'))
);

-- Assign next ID for new customer
CREATE SEQUENCE cur_customer_val START WITH 1 INCREMENT BY 1;

CREATE OR REPLACE TRIGGER new_customer_id
BEFORE INSERT ON customers
FOR EACH ROW
BEGIN
```

```
:NEW.customer_id := cur_customer_val.NEXTVAL;
END;
/

Prompt *** Created customers table ***
Prompt

-----

Prompt ***** # categories # *****
Rem menus.category_id reference this table.

CREATE TABLE categories (
    category_id NUMBER CONSTRAINT categories_category_id_pk PRIMARY
KEY,
    name VARCHAR2(30) CONSTRAINT categories_name_nn NOT NULL,
    description VARCHAR2(1000)
);

-- Assign next ID for new category
CREATE SEQUENCE cur_category_val START WITH 1 INCREMENT BY 1;

CREATE OR REPLACE TRIGGER new_category_ids
BEFORE INSERT ON categories
FOR EACH ROW
BEGIN
    :NEW.category_id := cur_category_val.NEXTVAL;
END;
/

Prompt *** Created categories table ***
Prompt

-----

Prompt ***** # promotions # *****
Rem orders.promotion_id reference this table.

CREATE TABLE promotions (
    promotion_id NUMBER CONSTRAINT promotions_promotion_id_pk PRIMARY KEY,
    name VARCHAR2(30) CONSTRAINT promotions_name_nn NOT NULL,
    discount_type VARCHAR2(2) CONSTRAINT promotions_discount_type_nn NOT NULL,
    discount_value NUMBER(6,2) CONSTRAINT promotions_discount_value_nn NOT
NULL,
    start_date DATE CONSTRAINT promotions_start_date_nn NOT NULL,
    end_date DATE CONSTRAINT promotions_end_date_nn NOT NULL,
```

UCCD2303 Group Assignment Question
February 2025 Trimester

```
CONSTRAINT chk_promotions_discount_type CHECK (discount_type IN ('%',  
'RM')),  
CONSTRAINT chk_promotions_discount_value CHECK (discount_value > 0.00),  
CONSTRAINT chk_promotions_end_date CHECK (end_date > start_date)  
);  
  
-- Assign next ID for new promotion  
CREATE SEQUENCE cur_promotion_val START WITH 1 INCREMENT BY 1;  
  
CREATE OR REPLACE TRIGGER new_promotion_id  
BEFORE INSERT ON promotions  
FOR EACH ROW  
BEGIN  
    :NEW.promotion_id := cur_promotion_val.NEXTVAL;  
END;  
/  
  
Prompt *** Created promotions table ***  
Prompt  
  
-----  
  
Prompt ***** # vouchers # *****  
Rem members_vouchers.voucher_code reference this table.  
  
CREATE TABLE vouchers (  
    -- Can be generated automatically or manually.  
    voucher_code VARCHAR2(15) CONSTRAINT vouchers_voucher_code_pk PRIMARY  
KEY,  
    name VARCHAR(30) CONSTRAINT vouchers_name_nn NOT  
NULL,  
    description  
VARCHAR2(500),  
    start_date  
DATE,  
    end_date DATE CONSTRAINT vouchers_end_date_nn NOT  
NULL,  
    min_spend NUMBER(4) DEFAULT 0 CONSTRAINT vouchers_min_spend_nn NOT NULL,  
    -- 100% rate for Fixed amount discount voucher  
    discount_rate NUMBER(3,2) CONSTRAINT vouchers_discount_rate_nn NOT  
NULL,  
    discount_cap NUMBER(6, 2) CONSTRAINT vouchers_discount_cap_nn NOT NULL,  
    membership_tier NUMBER(1) DEFAULT 1 CONSTRAINT vouchers_membership_tier_nn  
NOT NULL,  
    redeem_point NUMBER CONSTRAINT vouchers_redeem_points_nn NOT NULL,  
    CONSTRAINT chk_vouchers_end_date CHECK (end_date > start_date),  
    CONSTRAINT chk_vouchers_min_spend CHECK (min_spend >= 0),
```


UCCD2303 Group Assignment Question
February 2025 Trimester

```
CONSTRAINT chk_vouchers_discount_rate CHECK (discount_rate > 0.00 AND
discount_rate <= 1.00),
CONSTRAINT chk_vouchers_discount_cap CHECK (discount_cap > 0.00),
CONSTRAINT chk_vouchers_redeem_point CHECK (redeem_point >= 0),
CONSTRAINT chk_vouchers_membership_tier_limit CHECK (membership_tier <= 6)
);

CREATE OR REPLACE TRIGGER new_voucher
BEFORE INSERT ON vouchers
FOR EACH ROW
BEGIN
    :NEW.start_date := SYSDATE;
END;
/

Prompt *** Created vouchers table ***
Prompt

-----

-----

Rem Weak Entity 2nd : menus_in_branches, orders
-----

Prompt ***** # members_vouchers # *****
Rem orders.voucher_code reference this table.
Rem This table reference vouchers.voucher_code and members.member_id.

CREATE TABLE members_vouchers (
    member_id NUMBER CONSTRAINT members_vouchers_member_id_nn NOT NULL,
    voucher_code VARCHAR2(15) CONSTRAINT members_vouchers_voucher_code_nn NOT
NULL,
    leftover NUMBER DEFAULT 1 CONSTRAINT members_vouchers_leftover NOT NULL,
    CONSTRAINT members_vouchers_member_id_voucher_code_pk PRIMARY KEY
(member_id, voucher_code),
    CONSTRAINT members_vouchers_members_member_id_fk FOREIGN KEY (member_id)
REFERENCES members(member_id),
    CONSTRAINT members_vouchers_vouchers_voucher_code_fk FOREIGN KEY
(voucher_code) REFERENCES vouchers(voucher_code),
    CONSTRAINT chk_members_vouchers_leftover CHECK (leftover >= 0)
);

Prompt *** Created members_vouchers table ***
Prompt
```

```
-----  
-----  
  
Prompt ***** # menus # *****  
Rem items.item_id reference this table for inheritance.  
Rem orders_menus.item_id and menus_in_branches.menu_id reference this table.  
Rem This table reference categories.category_id  
  
CREATE TABLE menus (  
    menu_id NUMBER CONSTRAINT menus_menu_id_pk PRIMARY  
KEY,  
    category_id NUMBER CONSTRAINT menus_category_id_nn NOT  
NULL,  
    name VARCHAR2(30) CONSTRAINT menus_name_nn NOT NULL,  
    price NUMBER(6,2) CONSTRAINT menus_price_nn NOT NULL,  
    description VARCHAR2(1000),  
    type VARCHAR2(10) CONSTRAINT menus_type_nn NOT NULL,  
    CONSTRAINT menus_categories_category_id_fk FOREIGN KEY (category_id)  
REFERENCES categories(category_id),  
    CONSTRAINT chk_menus_price CHECK (price > 0)  
);  
  
-- Assign next ID for new menu menu item  
CREATE SEQUENCE cur_menu_val START WITH 1 INCREMENT BY 1;  
  
CREATE OR REPLACE TRIGGER new_menu_id  
BEFORE INSERT ON menus  
FOR EACH ROW  
BEGIN  
    :NEW.menu_id := cur_menu_val.NEXTVAL;  
END;  
/  
  
Prompt *** Created menus table ***  
Prompt  
  
-----  
-----  
  
Prompt ***** # items # *****  
Rem foods.item_id, drinks.item_id and merch.item_id reference this table as  
for Overlapping Inheritance.  
Rem This table reference menus.menu_id for inheritance.  
  
CREATE TABLE items (  
    item_id NUMBER CONSTRAINT items_item_id_pk PRIMARY KEY,  
    name VARCHAR2(30) CONSTRAINT items_name_nn NOT NULL
```

```
);

-- Assign next ID for new item
CREATE SEQUENCE cur_item_val START WITH 1 INCREMENT BY 1;

CREATE OR REPLACE TRIGGER new_item_id
BEFORE INSERT ON items
FOR EACH ROW
BEGIN
    :NEW.item_id := cur_item_val.NEXTVAL;
END;
/

Prompt *** Created items table ***
Prompt

-----

-----

Rem ***** inheritance for menus table through items
*****

-----

Prompt ***** # foods # *****
Rem This table reference items.item_id

CREATE TABLE foods (
    food_id VARCHAR(6) CONSTRAINT foods_food_id_pk PRIMARY
KEY,
    item_id NUMBER CONSTRAINT foods_item_id_nn NOT NULL,
    calorie NUMBER CONSTRAINT foods_calorie_nn NOT NULL,
    ingredient VARCHAR2(30) CONSTRAINT foods_ingredient_nn NOT NULL,
    CONSTRAINT foods_items_item_id_fk FOREIGN KEY (item_id) REFERENCES
items(item_id),
    CONSTRAINT chk_foods_calorie CHECK (calorie >= 0)
);

-- Assign next ID for new food
CREATE SEQUENCE cur_food_val START WITH 1 INCREMENT BY 1;

CREATE OR REPLACE TRIGGER new_food_id
BEFORE INSERT ON foods
FOR EACH ROW
BEGIN
    -- Assign new food_id = 'F' + next food ID with left padding
```

```
:NEW.food_id := 'F' || LPAD(cur_food_val.NEXTVAL, 5, '0');
END;
/

Prompt *** Created foods table ***
Prompt

-----

Prompt ***** # drinks # *****
Rem This table reference items.item_id

CREATE TABLE drinks (
    drink_id VARCHAR2(6) CONSTRAINT drinks_drink_id_pk PRIMARY
KEY,
    item_id NUMBER CONSTRAINT drinks_item_id_nn NOT NULL,
    calorie NUMBER CONSTRAINT drins_calorie_nn NOT NULL,
    available_size VARCHAR2(30) DEFAULT 'medium' CONSTRAINT
drinks_available_size_nn NOT NULL,
    CONSTRAINT drinks_items_item_id_fk FOREIGN KEY (item_id) REFERENCES
items(item_id),
    CONSTRAINT chk_drinks_calorie CHECK (calorie >= 0),
    CONSTRAINT chk_drinks_available_size CHECK (LOWER(available_size) IN
('small', 'medium', 'large', 'giant'))
);

-- Assign next ID for new drink
CREATE SEQUENCE cur_drink_val START WITH 1 INCREMENT BY 1;

CREATE OR REPLACE TRIGGER new_drink_id
BEFORE INSERT ON drinks
FOR EACH ROW
BEGIN
    -- Assign new drink = 'D' + next drink ID with left padding
    :NEW.drink_id := 'D' || LPAD(cur_drink_val.NEXTVAL, 5, '0');
END;
/

Prompt *** Created drinks table ***
Prompt

-----

Prompt ***** # merch # *****
Rem This table reference items.item_id
```

```
CREATE TABLE merch (
    merch_id VARCHAR2(6) CONSTRAINT merch_merch_id_pk PRIMARY KEY,
    item_id NUMBER CONSTRAINT merch_item_id_nn NOT NULL,
    type VARCHAR2(12) CONSTRAINT merch_type_nn NOT NULL,
    CONSTRAINT merch_items_item_id_fk FOREIGN KEY (item_id) REFERENCES
items(item_id)
);

-- Assign next ID for new merch
CREATE SEQUENCE cur_merch_val START WITH 1 INCREMENT BY 1;

CREATE OR REPLACE TRIGGER new_merch_id
BEFORE INSERT ON merch
FOR EACH ROW
BEGIN
    -- Assign new merch_id = 'F' + next merch ID with left padding
    :NEW.merch_id := 'M' || LPAD(cur_merch_val.NEXTVAL, 5, '0');
END;
/

Prompt *** Created merch table ***
Prompt

-----
-----
Rem ***** end of inheritance
*****
-----
-----
-----
-----

Prompt ***** # staffs_roles # *****
Rem This table reference staffs.staff_id and roles.role_id reference this
table for junction table.

CREATE TABLE staffs_roles (
    staff_id VARCHAR2(6),
    role_id VARCHAR2(6),
    CONSTRAINT staffs_roles_staff_id_role_id_pk PRIMARY KEY (staff_id,
role_id),
    CONSTRAINT staffs_roles_staffs_staff_id_fk FOREIGN KEY (staff_id)
REFERENCES staffs(staff_id),
    CONSTRAINT staffs_roles_roles_role_id_fk FOREIGN KEY (role_id) REFERENCES
roles(role_id)
);
```

UCCD2303 Group Assignment Question
February 2025 Trimester

```
Prompt *** Created staffs_roles table ***
Prompt

-----

Prompt ***** # branches # *****
Rem This table reference staffs.staff_id for manager_id.

CREATE TABLE branches (
    branch_id VARCHAR2(6) CONSTRAINT branches_branch_id_pk PRIMARY
KEY,
    manager_id VARCHAR2(6) CONSTRAINT branches_manager_id_nn NOT NULL,
    address VARCHAR2(300) CONSTRAINT branches_address_nn NOT NULL,
    CONSTRAINT branches_staffs_manager_id_fk FOREIGN KEY (manager_id)
REFERENCES staffs(staff_id)
);

-- Assign next ID for new branch
CREATE SEQUENCE cur_branch_val START WITH 1 INCREMENT BY 1;

CREATE OR REPLACE TRIGGER new_branch_id
BEFORE INSERT ON branches
FOR EACH ROW
BEGIN
    -- Assign new branch = 'B' + next branch ID with left padding
    :NEW.branch_id := 'B' || LPAD(cur_branch_val.NEXTVAL, 5, '0');
END;
/

Prompt *** Created branches table ***
Prompt

-----

Prompt ***** # menus_in_branches # *****
Rem This table reference menus.menu_id and branches.branch_id.

CREATE TABLE menus_in_branches (
    menu_id NUMBER,
    branch_id VARCHAR2(6),
    stock NUMBER(5) DEFAULT 0,
    CONSTRAINT menus_in_branches_menu_id_branch_id_pk PRIMARY KEY (menu_id,
branch_id),
    CONSTRAINT menus_in_branches_menus_menu_id_fk FOREIGN KEY (menu_id)
REFERENCES menus(menu_id),
```

```
        CONSTRAINT menus_in_branches_branches_branch_id_fk FOREIGN KEY (branch_id)
REFERENCES branches(branch_id)
);

Prompt *** Created menus_in_branches table ***
Prompt

-----

Prompt ***** # menus_items # *****
Rem This table reference menus.menu_id and items.item_id.

CREATE TABLE menus_items (
    menu_id NUMBER,
    item_id NUMBER,
    quantity NUMBER DEFAULT 0 CONSTRAINT menus_items_quantity_nn NOT NULL,
    CONSTRAINT menus_items_menu_id_item_id_pk PRIMARY KEY (menu_id, item_id),
    CONSTRAINT menus_items_menus_menu_id_fk FOREIGN KEY (menu_id) REFERENCES
menus(menu_id),
    CONSTRAINT menus_items_menus_item_id_fk FOREIGN KEY (item_id) REFERENCES
items(item_id),
    CONSTRAINT chk_menus_items_quantity CHECK (quantity >= 0)
);

Prompt *** Created menus_items table ***
Prompt

-----

Prompt ***** # orders # *****
Rem This table reference staffs.staff_id, customers.customer_id,
promotions.promotion_id, branches.branch_id and vouchers.voucher_code
Rem orders_menus.order_id and payments.order_id reference this table.

CREATE TABLE orders (
    order_id NUMBER CONSTRAINT orders_order_id_pk PRIMARY KEY,
    staff_handle VARCHAR2(6) CONSTRAINT orders_staff_handle_nn NOT NULL,
    customer_id NUMBER CONSTRAINT orders_customer_id_nn NOT NULL,
    member_id NUMBER,
    branch_id VARCHAR2(6) CONSTRAINT orders_branch_id_nn NOT NULL,
    promotion_id NUMBER,
    voucher_code VARCHAR2(15),
    order_type VARCHAR2(10) CONSTRAINT orders_order_type_nn NOT NULL,
    total_amount NUMBER(6,2) CONSTRAINT orders_total_amount_nn NOT NULL,
    discount_amount NUMBER(6,2) DEFAULT 0,
```

UCCD2303 Group Assignment Question
February 2025 Trimester

```
    order_datetime DATE DEFAULT SYSDATE CONSTRAINT orders_order_datetime_nn
NOT NULL,
    status NUMBER(1) DEFAULT 0,
    -- loyalty_points NUMBER(10) DEFAULT 0,
    CONSTRAINT orders_staffs_staff_handle_fk FOREIGN KEY (staff_handle)
REFERENCES staffs(staff_id),
    CONSTRAINT orders_customers_customer_id_fk FOREIGN KEY (customer_id)
REFERENCES customers(customer_id),
    CONSTRAINT orders_branches_branch_id_fk FOREIGN KEY (branch_id) REFERENCES
branches(branch_id),
    CONSTRAINT orders_promotions_promotion_id_fk FOREIGN KEY (promotion_id)
REFERENCES promotions(promotion_id),
    CONSTRAINT orders_members_vouchers_member_id_voucher_code_fk FOREIGN KEY
(member_id, voucher_code) REFERENCES members_vouchers(member_id,
voucher_code),
    CONSTRAINT chk_orders_status CHECK (status >= -1 AND status <= 4)
);

-- Assign next ID for new order
CREATE SEQUENCE cur_order_val START WITH 1 INCREMENT BY 1;

CREATE OR REPLACE TRIGGER new_order_id
BEFORE INSERT ON orders
FOR EACH ROW
BEGIN
    :NEW.order_id := cur_order_val.NEXTVAL;
END;
/

Prompt *** Created orders table ***
Prompt

-----
-----

-----
-----

Rem Weak Entity 3rd : orders_menus, payments
-----
-----

Prompt ***** # orders_menus # *****
Rem This table reference orders.order_id and menus.menu_id to form junction
table

CREATE TABLE orders_menus (
    order_id NUMBER,
```



```
menu_id NUMBER,
price_on_order NUMBER(6, 2) CONSTRAINT orders_menus_price_on_order_nn NOT
NULL,      -- For recording the price change
quantity NUMBER DEFAULT 1 CONSTRAINT orders_menus_quantity_nn NOT NULL,
CONSTRAINT orders_menus_order_id_pk PRIMARY KEY (order_id, menu_id),
CONSTRAINT orders_menus_orders_order_id_fk FOREIGN KEY (order_id)
REFERENCES orders(order_id),
CONSTRAINT orders_menus_menus_menu_id_fk FOREIGN KEY (menu_id) REFERENCES
menus(menu_id),
CONSTRAINT chk_orders_menus_quantity CHECK (quantity >= 1),
CONSTRAINT chk_orders_menus_price_on_order CHECK (price_on_order >= 0)
);

-- Retrieve the current price of the item into price_on_order
CREATE OR REPLACE TRIGGER set_price_on_order
BEFORE INSERT ON orders_menus
FOR EACH ROW
BEGIN
    SELECT price INTO :NEW.price_on_order
    FROM menus
    WHERE menu_id = :NEW.menu_id;
END;
/

Prompt *** Created orders_menus table ***
Prompt

-----

Prompt ***** # payments # *****
Rem This table reference orders.order_id

CREATE TABLE payments (
    payment_id NUMBER CONSTRAINT payments_payment_id_pk PRIMARY KEY,
    order_id NUMBER CONSTRAINT payments_order_id_nn NOT NULL,
    amount_paid NUMBER (6,2) CONSTRAINT payments_amount_paid_nn NOT NULL,
    payment_method VARCHAR2(15) CONSTRAINT payments_payment_method_nn NOT
NULL,
    payment_datetime DATE DEFAULT SYSDATE CONSTRAINT
payments_payment_datetime_nn NOT NULL,
    CONSTRAINT payments_orders_order_id_fk FOREIGN KEY (order_id) REFERENCES
orders(order_id),
    CONSTRAINT chk_payments_amount_paid CHECK(amount_paid >= 0)
);

-- Assign next ID for new payment
CREATE SEQUENCE cur_payment_val START WITH 1 INCREMENT BY 1;
```

```
CREATE OR REPLACE TRIGGER new_payment_info
BEFORE INSERT ON payments
FOR EACH ROW
BEGIN
    :NEW.payment_id := cur_payment_val.NEXTVAL;
    :NEW.payment_datetime := SYSTIMESTAMP;
END;
/

Prompt *** Created payments table ***
Prompt

-----
-----
-----
-----
Prompt ~ ~ ~ Tables Creation Completed ~ ~ ~
Prompt
-----
-----

-- Sample records for staffs table

INSERT INTO staffs (first_name, last_name, username, password, email, phone)
VALUES ('John', 'Doe', 'jdoe', 'Pass@123', 'jdoe@example.com', '0112345678');

INSERT INTO staffs (first_name, last_name, username, password, email, phone)
VALUES ('Jane', 'Smith', 'jsmith', 'Pass@456', 'jsmith@example.com',
'0123456789');

INSERT INTO staffs (first_name, last_name, username, password, email, phone)
VALUES ('Alice', 'Brown', 'abrown', 'Pass@789', 'abrown@example.com',
'0198765432');

INSERT INTO staffs (first_name, last_name, username, password, email, phone)
VALUES ('Bob', 'White', 'bwhite', 'Pass@321', 'bwhite@example.com',
'0187654321');

INSERT INTO staffs (first_name, last_name, username, password, email, phone)
VALUES ('Charlie', 'Davis', 'cdavis', 'Pass@654', 'cdavis@example.com',
'0176543210');

INSERT INTO staffs (first_name, last_name, username, password, email, phone)
VALUES ('Daniel', 'Lee', 'dlee', 'Pass@987', 'dlee@example.com',
'0161234567');
```

```
INSERT INTO staffs (first_name, last_name, username, password, email, phone)
VALUES ('Emma', 'Wilson', 'ewilson', 'Pass@654', 'ewilson@example.com',
'0172345678');

INSERT INTO staffs (first_name, last_name, username, password, email, phone)
VALUES ('Michael', 'Anderson', 'manderson', 'Pass@321',
'manderson@example.com', '0193456789');

INSERT INTO staffs (first_name, last_name, username, password, email, phone)
VALUES ('Sophia', 'Thomas', 'stthomas', 'Pass@147', 'stthomas@example.com',
'0114567890');

INSERT INTO staffs (first_name, last_name, username, password, email, phone)
VALUES ('Ethan', 'Harris', 'eharris', 'Pass@258', 'eharris@example.com',
'0125678901');

-- Sample records for roles table

INSERT INTO roles (role_id, name, description)
VALUES ('MNG', 'Manager', 'Branches operations and staff management');

INSERT INTO roles (role_id, name, description)
VALUES ('CHE', 'Chef', 'Prepares food and manages kitchen staff');

INSERT INTO roles (role_id, name, description)
VALUES ('CAS', 'Cashier', 'Handles customer transactions and billing');

INSERT INTO roles (role_id, name, description)
VALUES ('WAI', 'Waiter', 'Serves food and interacts with customers');

INSERT INTO roles (role_id, name, description)
VALUES ('CLE', 'Cleaner', 'Maintains cleanliness of the restaurant');

-- Sample records for members table

INSERT INTO members (first_name, last_name, username, password, email, phone,
address, date_of_birth, membership_tier, loyalty_points)
VALUES ('Emily', 'Clark', 'EmClark01', 'Pass@123', 'eclark@example.com',
'0131231234', '25, Jalan Anggerik 3, Taman Melati, 53100 Kuala Lumpur',
TO_DATE('1990-05-10', 'YYYY-MM-DD'), 3, 0);

INSERT INTO members (first_name, last_name, username, password, email, phone,
address, date_of_birth, membership_tier, loyalty_points)
```

```
VALUES ('Daniel', 'Johnson', 'JohnD02', 'Pass@456', 'djohnson@example.com',
'0142342345', '10, Lorong Merpati 2, Bandar Baru Uda, 81200 Johor Bahru,
Johor', TO_DATE('1985-08-15', 'YYYY-MM-DD'), 2, 150);

INSERT INTO members (first_name, last_name, username, password, email, phone,
address, date_of_birth, membership_tier, loyalty_points)
VALUES ('Sophia', 'Miller', 'MillerS03', 'Pass@789', 'smiller@example.com',
'0153453456', '88, Jalan Kenanga, Taman Indah, 75200 Melaka', TO_DATE('1998-
12-22', 'YYYY-MM-DD'), 1, 50);

INSERT INTO members (first_name, last_name, username, password, email, phone,
address, date_of_birth, membership_tier, loyalty_points)
VALUES ('Michael', 'Wilson', 'WilsonM04', 'Pass@321', 'mwilson@example.com',
'0164564567', '5, Persiaran Mawar, Seksyen 9, 40000 Shah Alam, Selangor',
TO_DATE('1992-03-30', 'YYYY-MM-DD'), 3, 300);

INSERT INTO members (first_name, last_name, username, password, email, phone,
address, date_of_birth, membership_tier, loyalty_points)
VALUES ('Olivia', 'Martinez', 'Martinez005', 'Pass@654',
'omartinez@example.com', '0175675678', '17, Jalan Rusa 4, Taman Bukit Jaya,
81300 Skudai, Johor', TO_DATE('1989-07-05', 'YYYY-MM-DD'), 4, 500);

-- Sample records for customers table

INSERT INTO customers (member_id, type)
VALUES (1, 'Member');

INSERT INTO customers (member_id, type)
VALUES (NULL, 'Guest');

INSERT INTO customers (member_id, type)
VALUES (2, 'Member');

INSERT INTO customers (member_id, type)
VALUES (NULL, 'Guest');

INSERT INTO customers (member_id, type)
VALUES (3, 'Member');

-- Sample records for categories table

INSERT INTO categories (name, description)
VALUES ('Appetizers', 'Start your meal with a delicious appetizer.');
```

```
INSERT INTO categories (name, description)
VALUES ('Main Course', 'Delicious burgers and fried chicken meals.');
```

```
INSERT INTO categories (name, description)
VALUES ('Desserts', 'Sweet treats to complete your meal.');
```

```
INSERT INTO categories (name, description)
VALUES ('Beverages', 'Refreshing drinks to pair with your food.');
```

```
INSERT INTO categories (name, description)
VALUES ('Combo Meals', 'Perfectly paired meal sets with drinks and sides.');
```

```
INSERT INTO categories (name, description)
VALUES ('Merch', 'Branded fast food merchandise including apparel,
collectibles, and accessories.');
```



```
-- Sample records for promotions table
```

```
INSERT INTO promotions (name, discount_type, discount_value, start_date,
end_date)
VALUES ('Summer Sale', '%', 15.00, TO_DATE('2025-06-01', 'YYYY-MM-DD'),
TO_DATE('2025-06-30', 'YYYY-MM-DD'));
```

```
INSERT INTO promotions (name, discount_type, discount_value, start_date,
end_date)
VALUES ('Sunny Season Savings', 'RM', 2.00, TO_DATE('2025-4-01', 'YYYY-MM-
DD'), TO_DATE('2025-12-25', 'YYYY-MM-DD'));
```

```
INSERT INTO promotions (name, discount_type, discount_value, start_date,
end_date)
VALUES ('Festive Price Slash', '%', 3.00, TO_DATE('2025-04-01', 'YYYY-MM-DD'),
TO_DATE('2025-05-07', 'YYYY-MM-DD'));
```

```
INSERT INTO promotions (name, discount_type, discount_value, start_date,
end_date)
VALUES ('Weekend Deal', 'RM', 100.00, TO_DATE('2025-04-03', 'YYYY-MM-DD'),
TO_DATE('2025-04-05', 'YYYY-MM-DD'));
```

```
INSERT INTO promotions (name, discount_type, discount_value, start_date,
end_date)
VALUES ('Clearance Sale', '%', 50.00, TO_DATE('2025-07-01', 'YYYY-MM-DD'),
TO_DATE('2025-07-15', 'YYYY-MM-DD'));
```



```
-- Sample records for vouchers table
```

```
INSERT INTO vouchers (voucher_code, name, description, start_date, end_date,
min_spend, discount_rate, discount_cap, membership_tier, redeem_point)
```

UCCD2303 Group Assignment Question
February 2025 Trimester

```
VALUES ('VC001', 'Summer Promo', 'Discount on summer products', SYSDATE,
TO_DATE('2025-06-30', 'YYYY-MM-DD'), 20, 0.15, 5.00, 1, 200);

INSERT INTO vouchers (voucher_code, name, description, start_date, end_date,
min_spend, discount_rate, discount_cap, membership_tier, redeem_point)
VALUES ('VC002', 'Holiday Special', 'Year-end holiday discounts', SYSDATE,
TO_DATE('2025-12-31', 'YYYY-MM-DD'), 10, 0.30, 3.00, 1, 300);

INSERT INTO vouchers (voucher_code, name, description, start_date, end_date,
min_spend, discount_rate, discount_cap, membership_tier, redeem_point)
VALUES ('VC003', 'Flash Sale', 'Limited time flash sale discounts', SYSDATE,
TO_DATE('2025-05-07', 'YYYY-MM-DD'), 50, 0.10, 10.00, 2, 150);

INSERT INTO vouchers (voucher_code, name, description, start_date, end_date,
min_spend, discount_rate, discount_cap, membership_tier, redeem_point)
VALUES ('VC004', 'Weekend Deal', 'Special discounts for weekend shoppers',
SYSDATE, TO_DATE('2025-06-09', 'YYYY-MM-DD'), 20, 0.25, 10.00, 3, 250);

INSERT INTO vouchers (voucher_code, name, description, start_date, end_date,
min_spend, discount_rate, discount_cap, membership_tier, redeem_point)
VALUES ('VC005', 'Spring Fever', 'Discount on all spring menu items', SYSDATE,
TO_DATE('2025-05-31', 'YYYY-MM-DD'), 20, 0.18, 10.00, 4, 220);

-- Sample records for members_vouchers table

INSERT INTO members_vouchers (member_id, voucher_code, leftover)
VALUES (1, 'VC001', 1);

INSERT INTO members_vouchers (member_id, voucher_code, leftover)
VALUES (2, 'VC002', 2);

INSERT INTO members_vouchers (member_id, voucher_code, leftover)
VALUES (3, 'VC003', 3);

INSERT INTO members_vouchers (member_id, voucher_code, leftover)
VALUES (4, 'VC004', 4);

INSERT INTO members_vouchers (member_id, voucher_code, leftover)
VALUES (5, 'VC005', 5);

-- Sample records for menus table

-- Insert menu items for 'Appetizers'

INSERT INTO menus (category_id, name, price, description, type)
```

```
VALUES (1, 'Mozzarella Sticks', 5.99, 'Crispy fried mozzarella sticks served
with marinara sauce.', 'Ala-Carte');

INSERT INTO menus (category_id, name, price, description, type)
VALUES (1, 'Fries', 3.99, 'Crispy golden fries served with ketchup.', 'Ala-
Carte');

INSERT INTO menus (category_id, name, price, description, type)
VALUES (1, 'Onion Rings', 4.99, 'Golden crispy onion rings with dipping
sauce.', 'Ala-Carte');

-- Insert menu items for 'Main Course'

INSERT INTO menus (category_id, name, price, description, type)
VALUES (2, 'Cheeseburger', 8.99, 'Juicy beef patty with melted cheese,
lettuce, and tomato.', 'Ala-Carte');

INSERT INTO menus (category_id, name, price, description, type)
VALUES (2, 'Fried Chicken', 5.99, 'Crispy fried chicken.', 'Ala-Carte');

INSERT INTO menus (category_id, name, price, description, type)
VALUES (2, 'Grilled Chicken Sandwich', 7.99, 'Grilled chicken breast with
lettuce and mayo on a bun.', 'Ala-Carte');

-- Insert menu items for 'Desserts'

INSERT INTO menus (category_id, name, price, description, type)
VALUES (3, 'Chocolate Brownie', 4.49, 'Rich chocolate brownie served with
vanilla ice cream.', 'Ala-Carte');

INSERT INTO menus (category_id, name, price, description, type)
VALUES (3, 'Apple Pie', 3.99, 'Warm apple pie with a flaky crust.', 'Ala-
Carte');

INSERT INTO menus (category_id, name, price, description, type)
VALUES (3, 'Milkshake', 5.49, 'Thick and creamy milkshake available in
chocolate, vanilla, and strawberry.', 'Ala-Carte');

-- Insert menu items for 'Beverages'

INSERT INTO menus (category_id, name, price, description, type)
VALUES (4, 'Soda', 2.49, 'Refreshing carbonated drink available in various
flavors.', 'Ala-Carte');

INSERT INTO menus (category_id, name, price, description, type)
```

```
VALUES (4, 'Iced Tea', 2.99, 'Freshly brewed iced tea, sweetened or
unsweetened.', 'Ala-Carte');

INSERT INTO menus (category_id, name, price, description, type)
VALUES (4, 'Coffee', 1.99, 'Hot brewed coffee with optional cream and sugar.',
'Ala-Carte');

INSERT INTO menus (category_id, name, price, description, type)
VALUES (4, 'Lemonade', 3.49, 'Freshly squeezed lemonade with a hint of mint.',
'Ala-Carte');

INSERT INTO menus (category_id, name, price, description, type)
VALUES (4, 'Hot Chocolate', 3.99, 'Rich and creamy hot chocolate topped with
whipped cream.', 'Ala-Carte');

-- Insert menu items for 'Combo Meals'

INSERT INTO menus (category_id, name, price, description, type)
VALUES (5, 'Burger Combo', 10.99, 'Cheeseburger with fries and a soda.',
'Meal');

INSERT INTO menus (category_id, name, price, description, type)
VALUES (5, 'Fried Chicken Combo', 12.99, '2-piece fried chicken with fries and
a soda.', 'Meal');

INSERT INTO menus (category_id, name, price, description, type)
VALUES (5, 'Grilled Chicken Sandwich Combo', 11.49, 'Grilled chicken sandwich
with onion rings and an iced tea.', 'Meal');

-- Insert menu items for 'Merch'

INSERT INTO menus (category_id, name, price, description, type)
VALUES (6, 'Logo T-shirt', 19.99, 'Comfortable cotton t-shirt with the
restaurant logo.', 'Merch');

INSERT INTO menus (category_id, name, price, description, type)
VALUES (6, 'Collector's Cup', 9.99, 'Exclusive limited edition cup with the
restaurant mascot design.', 'Merch');

INSERT INTO menus (category_id, name, price, description, type)
VALUES (6, 'Restaurant Hat', 14.99, 'Stylish hat with the restaurant logo for
a casual look.', 'Merch');

INSERT INTO menus (category_id, name, price, description, type)
VALUES (6, 'Keychain', 4.99, 'Metal keychain with the restaurant branding.',
'Merch');
```



```
INSERT INTO menus (category_id, name, price, description, type)
VALUES (6, 'Mug', 7.99, 'Ceramic mug featuring the restaurant logo, perfect
for hot beverages.', 'Merch');
```

-- Sample records for items table

```
INSERT INTO items (item_id, name)
VALUES (1, 'Mozzarella Sticks');
```

```
INSERT INTO items (item_id, name)
VALUES (2, 'Fries');
```

```
INSERT INTO items (item_id, name)
VALUES (3, 'Onion Rings');
```

```
INSERT INTO items (item_id, name)
VALUES (4, 'Cheeseburger');
```

```
INSERT INTO items (item_id, name)
VALUES (5, 'Fried Chicken');
```

```
INSERT INTO items (item_id, name)
VALUES (6, 'Grilled Chicken Sandwich');
```

```
INSERT INTO items (item_id, name)
VALUES (7, 'Chocolate Brownie');
```

```
INSERT INTO items (item_id, name)
VALUES (8, 'Apple Pie');
```

```
INSERT INTO items (item_id, name)
VALUES (9, 'Milkshake');
```

```
INSERT INTO items (item_id, name)
VALUES (10, 'Soda');
```

```
INSERT INTO items (item_id, name)
VALUES (11, 'Iced Tea');
```

```
INSERT INTO items (item_id, name)
VALUES (12, 'Coffee');
```

```
INSERT INTO items (item_id, name)
VALUES (13, 'Lemonade');
```

```
INSERT INTO items (item_id, name)
```

```
VALUES (14, 'Hot Chocolate');

INSERT INTO items (item_id, name)
VALUES (15, 'Burger Combo');

INSERT INTO items (item_id, name)
VALUES (16, 'Fried Chicken Combo');

INSERT INTO items (item_id, name)
VALUES (17, 'Grilled Chicken Sandwich Combo');

INSERT INTO items (item_id, name)
VALUES (18, 'Logo T-shirt');

INSERT INTO items (item_id, name)
VALUES (19, 'Collector's Cup');

INSERT INTO items (item_id, name)
VALUES (20, 'Restaurant Hat');

INSERT INTO items (item_id, name)
VALUES (21, 'Keychain');

INSERT INTO items (item_id, name)
VALUES (22, 'Mug');

-- Sample records for foods table

INSERT INTO foods (item_id, calorie, ingredient)
VALUES (1, 400, 'Mozzarella Cheese');

INSERT INTO foods (item_id, calorie, ingredient)
VALUES (2, 400, 'Potatoes');

INSERT INTO foods (item_id, calorie, ingredient)
VALUES (3, 320, 'Onions');

INSERT INTO foods (item_id, calorie, ingredient)
VALUES (4, 600, 'Beef Patty');

INSERT INTO foods (item_id, calorie, ingredient)
VALUES (5, 500, 'Chicken');

INSERT INTO foods (item_id, calorie, ingredient)
VALUES (6, 480, 'Chicken Breast');

INSERT INTO foods (item_id, calorie, ingredient)
```

```
VALUES (7, 450, 'Chocolate');

INSERT INTO foods (item_id, calorie, ingredient)
VALUES (8, 350, 'Apple Filling');

INSERT INTO foods (item_id, calorie, ingredient)
VALUES (9, 500, 'Milk and Ice Cream');

-- Sample records for drinks table

-- Soda
INSERT INTO drinks (item_id, calorie, available_size)
VALUES (10, 150, 'medium');

-- Iced Tea
INSERT INTO drinks (item_id, calorie, available_size)
VALUES (11, 0, 'large');

-- Coffee
INSERT INTO drinks (item_id, calorie, available_size)
VALUES (12, 5, 'medium');

-- Lemonade
INSERT INTO drinks (item_id, calorie, available_size)
VALUES (13, 5, 'medium');

-- Hot Chocolate
INSERT INTO drinks (item_id, calorie, available_size)
VALUES (14, 5, 'medium');

-- Sample records for merch table

-- Example: T-shirt
INSERT INTO merch (item_id, type)
VALUES (18, 'Apparel');

-- Example: Collector's Cup
INSERT INTO merch (item_id, type)
VALUES (19, 'Collectible');

-- Example: Hat
INSERT INTO merch (item_id, type)
VALUES (20, 'Apparel');

-- Example: Keychain
INSERT INTO merch (item_id, type)
```

```
VALUES (21, 'Accessories');

-- Example: Mug
INSERT INTO merch (item_id, type)
VALUES (22, 'Drinkware');

-- Sample records for menus_items table

-- Ala-Carte items (each item matches itself)

-- Mozzarella Sticks
INSERT INTO menus_items (menu_id, item_id, quantity)
VALUES (1, 1, 1);

-- Fries
INSERT INTO menus_items (menu_id, item_id, quantity)
VALUES (2, 2, 1);

-- Onion Rings
INSERT INTO menus_items (menu_id, item_id, quantity)
VALUES (3, 3, 1);

-- Cheeseburger
INSERT INTO menus_items (menu_id, item_id, quantity)
VALUES (4, 4, 1);

-- Fried Chicken
INSERT INTO menus_items (menu_id, item_id, quantity)
VALUES (5, 5, 1);

-- Grilled Chicken Sandwich
INSERT INTO menus_items (menu_id, item_id, quantity)
VALUES (6, 6, 1);

-- Chocolate Brownie
INSERT INTO menus_items (menu_id, item_id, quantity)
VALUES (7, 7, 1);

-- Apple Pie
INSERT INTO menus_items (menu_id, item_id, quantity)
VALUES (8, 8, 1);

-- Milkshake
INSERT INTO menus_items (menu_id, item_id, quantity)
VALUES (9, 9, 1);

-- Soda
```

UCCD2303 Group Assignment Question
February 2025 Trimester

```
INSERT INTO menus_items (menu_id, item_id, quantity)
VALUES (10, 10, 1);

-- Iced Tea
INSERT INTO menus_items (menu_id, item_id, quantity)
VALUES (11, 11, 1);

-- Coffee
INSERT INTO menus_items (menu_id, item_id, quantity)
VALUES (12, 12, 1);

-- Lemonade
INSERT INTO menus_items (menu_id, item_id, quantity)
VALUES (13, 13, 1);

-- Hot Chocolate
INSERT INTO menus_items (menu_id, item_id, quantity)
VALUES (14, 14, 1);

-- Meal items (include all relevant sub-items)

-- Burger (part of Burger Combo)
INSERT INTO menus_items (menu_id, item_id, quantity)
VALUES (15, 4, 1);

-- Fries (part of Burger Combo)
INSERT INTO menus_items (menu_id, item_id, quantity)
VALUES (15, 2, 1);

-- Soda (part of Burger Combo)
INSERT INTO menus_items (menu_id, item_id, quantity)
VALUES (15, 10, 1);

-- Fried Chicken (part of Fried Chicken Combo)
INSERT INTO menus_items (menu_id, item_id, quantity)
VALUES (16, 5, 2);

-- Fries (part of Fried Chicken Combo)
INSERT INTO menus_items (menu_id, item_id, quantity)
VALUES (16, 2, 1);

-- Soda (part of Fried Chicken Combo)
INSERT INTO menus_items (menu_id, item_id, quantity)
VALUES (16, 10, 1);

-- Grilled Chicken Sandwich (part of Grilled Chicken Sandwich Combo)
INSERT INTO menus_items (menu_id, item_id, quantity)
```

```
VALUES (17, 6, 1);

-- Onion Rings (part of Grilled Chicken Sandwich Combo)
INSERT INTO menus_items (menu_id, item_id, quantity)
VALUES (17, 3, 1);

-- Iced Tea (part of Grilled Chicken Sandwich Combo)
INSERT INTO menus_items (menu_id, item_id, quantity)
VALUES (17, 11, 1);

-- Sample records for staffs_roles table

INSERT INTO staffs_roles (staff_id, role_id)
VALUES ('S00001', 'MNG');

INSERT INTO staffs_roles (staff_id, role_id)
VALUES ('S00002', 'MNG');

INSERT INTO staffs_roles (staff_id, role_id)
VALUES ('S00003', 'MNG');

INSERT INTO staffs_roles (staff_id, role_id)
VALUES ('S00004', 'MNG');

INSERT INTO staffs_roles (staff_id, role_id)
VALUES ('S00005', 'MNG');

INSERT INTO staffs_roles (staff_id, role_id)
VALUES ('S00006', 'CAS');

INSERT INTO staffs_roles (staff_id, role_id)
VALUES ('S00007', 'CAS');

INSERT INTO staffs_roles (staff_id, role_id)
VALUES ('S00008', 'CAS');

INSERT INTO staffs_roles (staff_id, role_id)
VALUES ('S00009', 'CAS');

INSERT INTO staffs_roles (staff_id, role_id)
VALUES ('S00010', 'CAS');

-- Sample records for branches table

INSERT INTO branches (manager_id, address)
VALUES ('S00001', '123 Main St, City Center, KL');
```

```
INSERT INTO branches (manager_id, address)
VALUES ('S00001', '456 Elm St, Uptown, Penang');

INSERT INTO branches (manager_id, address)
VALUES ('S00001', '789 Oak St, Suburban Area, JB');

INSERT INTO branches (manager_id, address)
VALUES ('S00001', '321 Pine St, Downtown, KL');

INSERT INTO branches (manager_id, address)
VALUES ('S00001', '654 Maple St, Waterfront, Melaka');

-- Sample records for menus_in_branches table

INSERT INTO menus_in_branches (menu_id, branch_id, stock)
VALUES (1, 'B00001', 50);

INSERT INTO menus_in_branches (menu_id, branch_id, stock)
VALUES (2, 'B00001', 30);

INSERT INTO menus_in_branches (menu_id, branch_id, stock)
VALUES (3, 'B00001', 30);

INSERT INTO menus_in_branches (menu_id, branch_id, stock)
VALUES (4, 'B00001', 0);

INSERT INTO menus_in_branches (menu_id, branch_id, stock)
VALUES (5, 'B00001', 40);

INSERT INTO menus_in_branches (menu_id, branch_id, stock)
VALUES (1, 'B00002', 45);

INSERT INTO menus_in_branches (menu_id, branch_id, stock)
VALUES (2, 'B00002', 30);

INSERT INTO menus_in_branches (menu_id, branch_id, stock)
VALUES (3, 'B00002', 30);

INSERT INTO menus_in_branches (menu_id, branch_id, stock)
VALUES (4, 'B00002', 30);

INSERT INTO menus_in_branches (menu_id, branch_id, stock)
VALUES (6, 'B00002', 30);
```

UCCD2303 Group Assignment Question
February 2025 Trimester

```
INSERT INTO menus_in_branches (menu_id, branch_id, stock)
VALUES (1, 'B00003', 45);

INSERT INTO menus_in_branches (menu_id, branch_id, stock)
VALUES (2, 'B00003', 30);

INSERT INTO menus_in_branches (menu_id, branch_id, stock)
VALUES (3, 'B00003', 30);

INSERT INTO menus_in_branches (menu_id, branch_id, stock)
VALUES (4, 'B00003', 30);

INSERT INTO menus_in_branches (menu_id, branch_id, stock)
VALUES (5, 'B00003', 20);

INSERT INTO menus_in_branches (menu_id, branch_id, stock)
VALUES (10, 'B00003', 30);

INSERT INTO menus_in_branches (menu_id, branch_id, stock)
VALUES (15, 'B00003', 50);

INSERT INTO menus_in_branches (menu_id, branch_id, stock)
VALUES (16, 'B00003', 60);

INSERT INTO menus_in_branches (menu_id, branch_id, stock)
VALUES (1, 'B00004', 60);

INSERT INTO menus_in_branches (menu_id, branch_id, stock)
VALUES (2, 'B00004', 30);

INSERT INTO menus_in_branches (menu_id, branch_id, stock)
VALUES (3, 'B00004', 30);

INSERT INTO menus_in_branches (menu_id, branch_id, stock)
VALUES (4, 'B00004', 30);

INSERT INTO menus_in_branches (menu_id, branch_id, stock)
VALUES (5, 'B00004', 30);

INSERT INTO menus_in_branches (menu_id, branch_id, stock)
VALUES (1, 'B00005', 40);

INSERT INTO menus_in_branches (menu_id, branch_id, stock)
VALUES (2, 'B00005', 30);

INSERT INTO menus_in_branches (menu_id, branch_id, stock)
```



```
VALUES (3, 'B00005', 30);

INSERT INTO menus_in_branches (menu_id, branch_id, stock)
VALUES (4, 'B00005', 40);

INSERT INTO menus_in_branches (menu_id, branch_id, stock)
VALUES (5, 'B00005', 30);

-- Sample records for orders table

INSERT INTO orders (staff_handle, customer_id, member_id, branch_id,
promotion_id, voucher_code, order_type, total_amount, discount_amount, status)
VALUES ('S00006', 1, 1, 'B00001', 2, NULL, 'Dine-In', 15.97, 2.34, 4);

INSERT INTO orders (staff_handle, customer_id, member_id, branch_id,
promotion_id, voucher_code, order_type, total_amount, discount_amount, status)
VALUES ('S00007', 2, 2, 'B00002', NULL, 'VC002', 'Takeaway', 20.97, 3, 4);

INSERT INTO orders (staff_handle, customer_id, member_id, branch_id,
promotion_id, voucher_code, order_type, total_amount, discount_amount, status)
VALUES ('S00008', 3, 3, 'B00003', 3, 'VC003', 'Delivery', 59.94, 7.79, -1);

INSERT INTO orders (staff_handle, customer_id, member_id, branch_id,
promotion_id, voucher_code, order_type, total_amount, discount_amount, status)
VALUES ('S00009', 4, 4, 'B00004', NULL, NULL, 'Takeaway', 5.99, 0.00, 1);

INSERT INTO orders (staff_handle, customer_id, member_id, branch_id,
promotion_id, voucher_code, order_type, total_amount, discount_amount, status)
VALUES ('S00010', 5, 5, 'B00005', NULL, NULL, 'Dine-In', 11.97, 0.00, 0);

-- Sample records for orders_menus table

-- Insert into orders_menus table for Order 1

-- Order 1 includes 2 of Menu Item 1
INSERT INTO orders_menus (order_id, menu_id, price_on_order, quantity)
VALUES (1, 1, 5.99, 2);

-- Order 1 includes 1 of Menu Item 2
INSERT INTO orders_menus (order_id, menu_id, price_on_order, quantity)
VALUES (1, 2, 3.99, 1);

-- Insert into orders_menus table for Order 2

-- Order 2 includes 1 of Menu Item 3
```

UCCD2303 Group Assignment Question
February 2025 Trimester

```
INSERT INTO orders_menus (order_id, menu_id, price_on_order, quantity)
VALUES (2, 3, 4.99, 1);

-- Order 2 includes 2 of Menu Item 6
INSERT INTO orders_menus (order_id, menu_id, price_on_order, quantity)
VALUES (2, 6, 7.99, 2);

-- Insert into orders_menus table for Order 3

-- Order 3 includes 2 of Menu Item 1
INSERT INTO orders_menus (order_id, menu_id, price_on_order, quantity)
VALUES (3, 1, 5.99, 2);

-- Order 3 includes 2 of Menu Item 15
INSERT INTO orders_menus (order_id, menu_id, price_on_order, quantity)
VALUES (3, 15, 10.99, 2);

-- Order 3 includes 3 of Menu Item 16
INSERT INTO orders_menus (order_id, menu_id, price_on_order, quantity)
VALUES (3, 16, 12.99, 2);

-- Insert into orders_menus table for Order 4

-- Order 4 includes 1 of Menu Item 1
INSERT INTO orders_menus (order_id, menu_id, price_on_order, quantity)
VALUES (4, 1, 5.99, 1);

-- Insert into orders_menus table for Order 5

-- Order 5 includes 3 of Menu Item 2
INSERT INTO orders_menus (order_id, menu_id, price_on_order, quantity)
VALUES (5, 2, 3.99, 3);

-- Sample records for payments table

INSERT INTO payments (payment_id, order_id, amount_paid, payment_method)
VALUES (1, 1, 13.63, 'Credit Card');

INSERT INTO payments (payment_id, order_id, amount_paid, payment_method)
VALUES (2, 2, 17.97, 'Cash');

INSERT INTO payments (payment_id, order_id, amount_paid, payment_method)
VALUES (3, 3, 52.15, 'Debit Card');

INSERT INTO payments (payment_id, order_id, amount_paid, payment_method)
```

UCCD2303 Group Assignment Question
February 2025 Trimester

```
VALUES (4, 4, 5.99, 'Mobile Payment');  
  
INSERT INTO payments (payment_id, order_id, amount_paid, payment_method)  
VALUES (5, 5, 11.97, 'Cash');
```